

Team-Agnostic MLB Pitcher K% Projection

An explainable, leakage-aware 2025 forecasting case study by Kyle Kinney.

Notebook setup

This analysis uses the reproducible Python environment defined in `environment.yml`. The notebook combines explanatory Markdown, executable Python code, and saved results in one document. Project-relative file paths allow it to run from either the project root or the `notebooks` folder.

```
# Confirm that the notebook has an active Python runtime without exposing local paths.  
print("Python runtime detected successfully.")
```

Python runtime detected successfully.

Objective and information cutoff

The objective of this project is to predict 2025 strikeout percentage for each of the 873 players in the modeling dataset. The workflow is team-agnostic: it scores the full player population and does not use team identity as a feature. Every performance predictor must have been available before 2025 Opening Day. The 2025 rows contribute only player identity and age, which is known before the season; 2025 K%, TBF, team, and Stuff+ are sealed until the final methodology and predictions are frozen.

Unweighted MAE (mean absolute error) is the primary metric because the modeling objective gives every player one prediction. RMSE (root mean squared error) measures sensitivity to large misses, and TBF-weighted MAE provides a secondary view emphasizing pitchers who faced more batters.

- **MAE:** A metric that tells us the mean absolute difference between the predicted values and the actual values in a dataset. The lower the MAE, the better a model fits a dataset.
- **RMSE:** A metric that tells us the square root of the average squared difference between the predicted values and the actual values in a dataset. The lower the RMSE, the better a model fits a dataset.

Analysis roadmap

1. Objective and information cutoff
2. Data validation
3. Exploratory analysis
4. Projection baselines
5. Feature engineering
6. Historical backtesting
7. Model comparison

8. Final 2025 predictions
9. Limitations and possible improvements
10. Reproducibility and citations

```
from pathlib import Path

import pandas as pd

# Locate the project folder whether the notebook starts here or in /notebooks.
project_root = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
data_path = project_root / "data" / "k_2026.csv"

# Load the player-season data without modifying the source file.
df = pd.read_csv(data_path)

# Verify the source, dimensions, sample rows, and inferred data types.
print(f>Data file: {data_path.relative_to(project_root)}")
print(f>Shape: {df.shape}")
display(df.head())
display(df.dtypes.to_frame("dtype"))
```

Data file: data\k_2026.csv

Shape: (4371, 8)

	Season	Name	PlayerId	Team	K%	TBF	Stuff+	Age
0	2021	Francisco Mejía	642336	TBR	0.0	14	-11.100000	25
1	2021	Josh Reddick	502210	ARI	0.0	5	-11.018824	34
2	2021	Albert Almora Jr.	546991	NYM	0.0	6	9.100000	27
3	2021	Anthony Bemboom	621532	LAA	0.0	7	15.120000	31
4	2021	Austin Wynns	642851	BAL	0.0	4	20.426667	30

dtype	
Season	int64
Name	str
PlayerId	int64
Team	str
K%	float64
TBF	int64
Stuff+	float64
Age	int64

Historical K% baselines

All predictors for season t use only seasons before t . The 2025 outcomes remain sealed until the methodology is finalized. Four transparent starting projections are considered:

1. **Prior-season K%:** the pitcher's K% in season $t-1$.
2. **Available-history K%:** strikeouts divided by TBF across all seasons available in the dataset before t .
3. **Recent K%:** the TBF-weighted K% across the preceding two seasons.
4. **Marcel-style K%:** the preceding three seasons receive pitcher recency weights of 3/2/1, are weighted by TBF, and are regressed toward the corresponding league K%.

Tom Tango designed Marcel as a deliberately simple minimum-competence projection: weight recent MLB seasons more heavily, regress small samples toward the league mean, and apply an age adjustment. This notebook calls the baseline Marcel-style because it adapts that framework specifically to pitcher K% and postpones age effects for separate testing. The amount of league-average regression is selected using only the 2022–2023 development forecasts, leaving 2024 untouched for final model validation.

Source: <https://tangotiger.net/marcel/>

This section creates and evaluates four simple methods for projecting future pitcher K% while ensuring that each historical prediction uses only information available at the time.

1. Define reusable calculation functions

The code defines functions that:

- Calculate a TBF-weighted K% for an entire group.
- Calculate a TBF-weighted K% for each player.
- Calculate TBF-weighted player metrics such as Stuff+.
- Create baselines and candidate features for one target season.
- Repeat that process across multiple historical seasons.

2. Create four historical K% baselines

For every player, the code creates:

1. **Prior-season K%** — uses only the immediately preceding season.
2. **Available-history K%** — uses all available seasons before the target year.
3. **Recent K%** — uses the previous two seasons.
4. **Marcel-style K%** — uses three seasons, recency weights, TBF, and league-average shrinkage.

3. Prevent future-data leakage

Before predicting a target season, the code removes that season and every later season from the available history.

For example:

- A 2022 projection uses information through 2021.
- A 2023 projection uses information through 2022.
- A 2024 projection uses information through 2023.
- A 2025 projection uses information through 2024.

4. Build the Marcel-style baseline

The Marcel calculation:

- Uses the previous three seasons.
- Applies recency weights of **3, 2, and 1**.
- Uses TBF so larger samples receive more influence.
- Pulls uncertain player estimates toward the league-average K%.
- Calculates a reliability value based on how much player history is available.

5. Select Marcel's shrinkage strength

The code tests several possible values:

```
REGRESSION_TBF_GRID = [50, 100, 200, 400, 800]
```

Each value represents imaginary TBF at the league-average K%.

The code:

- Tests each value on the 2022–2023 development forecasts.
- Calculates Marcel's MAE for each value.
- Selects the value with the lowest MAE.
- Chooses **400 pseudo-TBF** as the final shrinkage strength.
- Leaves 2024 untouched for later validation of the finalized Ridge model.

6. Compare the four baselines

After Marcel is finalized, the code compares all four baselines using:

- **MAE:** the average absolute prediction error.
- **RMSE:** an error measure that emphasizes large misses.
- **TBF-weighted MAE:** an error measure that gives higher-workload pitchers more influence.

This establishes how accurately simple forecasting methods can predict future K%.

7. Create candidate model features

The code also creates player-level features covering:

- Historical K%
- Workload

- Recency
- Marcel reliability
- Stuff+
- Stuff+ change
- Age
- Missing history
- League context

Missing player K% values are replaced with the prior-season league average, while missingness flags tell the model that the original player information was unavailable.

8. Prepare the 2025 feature table

Finally, the code creates `features_2025` using data available through 2024.

This table contains:

- All four baseline projections
- Workload features
- Stuff+ features
- Age features
- Missingness indicators
- League-context features

Marcel becomes the strongest simple benchmark and one candidate input for the later Ridge model. The Ridge stage tests whether combining the selected features predicts K% more accurately than Marcel alone.

```
# Import NumPy for numerical calculations.
# "np" is the standard shortened name used to reference NumPy.
import numpy as np

# List the historical seasons that will be treated as prediction targets.
# These historical predictions allow us to evaluate methods against known results.
BACKTEST_SEASONS = [2022, 2023, 2024]

# Use the previous two seasons when calculating the recent K% baseline.
RECENT_RATE_YEARS = 2
# Use up to three previous seasons in the Marcel-style calculation.
MARCEL_YEARS = 3

# Give more influence to more recent seasons in the Marcel calculation.
MARCEL_WEIGHTS = {1: 3.0, 2: 2.0, 3: 1.0}

# List the possible Marcel regression strengths that will be tested.
# Each number represents imaginary League-average TBF.
# Larger regression-TBF values pull projections more strongly toward the League average, espec
REGRESSION_TBF_GRID = [50, 100, 200, 400, 800]
```

```

# Keep only seasons through 2024 for model development.
# .loc selects rows meeting the condition inside the brackets.
# .copy creates an independent DataFrame that can be safely modified.
development_df = df.loc[df["Season"] <= 2024].copy()

# Select the players who need 2025 projections.
# Keep only identity and age because these are known before 2025 performance occurs, no 2025 k
# reset_index creates a clean row index beginning at zero.
# drop=True prevents the old index from becoming a new column.
players_2025 = df.loc[
    df["Season"] == 2025,
    ["PlayerId", "Name", "Age"]
].reset_index(drop=True)

# Confirm that the development data ends in 2024.
# assert stops the notebook with an error if this condition is false.
assert development_df["Season"].max() == 2024

# Confirm that every player appears only once in the 2025 prediction list (avoid duplicates).
assert players_2025["PlayerId"].is_unique

# Print the number of historical rows available for model development.
# len counts the number of rows.
# :, adds commas to large numbers when displayed.
print(f"Development rows: {len(development_df):,}")

# Print the number of players requiring 2025 projections.
print(f"Players to project for 2025: {len(players_2025):,}")

```

Development rows: 3,498
 Players to project for 2025: 873

```

# Define a reusable function that calculates a group's TBF-weighted K%.
# This avoids repeating the formula whenever a historical league-average K% is needed.
# "frame" is the DataFrame supplied when the function is called.
def weighted_k_pct(frame):
    # Add the TBF from every row to calculate the group's total workload.
    total_tbf = frame["TBF"].sum()
    # Return a missing value when there are no TBF.
    # Otherwise, estimate total strikeouts and divide by total TBF.
    # This weights each row according to its workload.
    return np.nan if total_tbf == 0 else (frame["K%"] * frame["TBF"]).sum() / total_tbf

# Define a reusable function that calculates one TBF-weighted K% per player.
# "output_name" lets the same function create differently named baseline results, such as price.
def player_rate(frame, output_name):
    # Create a temporary _strikeouts column by multiplying each row's K% by TBF.
    # This reconstructs strikeouts because the dataset provides K% rather than strikeout total
    totals = (
        frame.assign(_strikeouts=frame["K%"] * frame["TBF"])
        # Group rows belonging to the same player, allowing multiple seasons or rows to be con

```

```

        .groupby("PlayerId", as_index=True)
        # Add each player's estimated strikeouts and TBF across the selected rows.
        # Both totals are needed to combine K% correctly across different workloads.
        .agg(
            _strikeouts=("_strikeouts", "sum"),
            _tbf=("TBF", "sum")
        )
    )
    # Divide each player's estimated strikeouts by their total TBF.
    # Rename the result so we know which historical baseline it represents.
    return (
        totals["_strikeouts"] / totals["_tbf"]
    ).rename(output_name)

```

Define a reusable function that calculates each player's TBF-weighted average for a selected
"value_column" identifies the metric being averaged.
"output_name" labels the returned result.

```

def player_weighted_mean(frame, value_column, output_name):
    # Keep only rows where the selected metric exists and TBF is positive.
    # Missing values cannot be averaged, and zero TBF provides no usable weight.
    valid = frame.loc[
        frame[value_column].notna() & (frame["TBF"] > 0)
    ].copy()
    # Multiply the metric by TBF so larger samples influence the average more.
    valid["_weighted_value"] = valid[value_column] * valid["TBF"]
    # Combine the weighted metric totals and TBF separately for each player.
    totals = valid.groupby("PlayerId").agg(
        _weighted_value=("_weighted_value", "sum"),
        _weight=("TBF", "sum")
    )
    # Divide each player's weighted total by their total TBF.
    # Rename the result to describe the metric that was calculated.
    return (
        totals["_weighted_value"] / totals["_weight"]
    ).rename(output_name)

```

Define a reusable function that creates the historical K% baselines and candidate model feat
history: the historical data available for creating predictors
target_players: the players who need predictions
target_season: the season being predicted
regression_tbf: the amount of imaginary league-average average TBF used to pull Marcel proje

```

def build_baselines(history, target_players, target_season, regression_tbf):
    # Keep only data from seasons before the season being predicted.
    # This prevents actual target-season performance from leaking into its prediction.
    history = history.loc[history["Season"] < target_season].copy()
    # Create one prediction row for each player in the target season.
    targets = (
        # Keep only the player information known before performance occurs.
        target_players[["PlayerId", "Name", "Age"]]
        # Ensure each player appears only once in the prediction table.
        .drop_duplicates("PlayerId")
        # Replace the old row index with a clean index beginning at zero.
        .reset_index(drop=True)
    )

```

)

```
# Keep only the season immediately before the target season.
# Variable used for the prior-season K% baseline.
prior_year = history.loc[history["Season"] == target_season - 1]

# Keep only the season two years before the target season.
# Variable allows comparisons with the prior year, such as changes in Stuff+.
two_years_ago = history.loc[history["Season"] == target_season - 2]

# Keep the previous two seasons.
# Variable is used to calculate the recent K% baseline.
recent_rate_history = history.loc[history["Season"] >= target_season - RECENT_RATE_YEARS]

# Keep the previous three seasons.
# Variable (Marcel) uses this history to emphasize recent performance while retaining more
marcel_history = history.loc[history["Season"] >= target_season - MARCEL_YEARS]

# Sort each player's history chronologically.
# Group rows by player and keep their latest available observation.
# Variable still finds recent information when a player missed the immediately prior season.
# Set playerId as the index so values can later be matched to target players.
latest = history.sort_values(["PlayerId", "Season"]).groupby("PlayerId").tail(1).set_index("PlayerId")

# Calculate the league-wide K% from the season immediately before the target year.
# This provides league context using only information available before the prediction.
league_prior_k_pct = weighted_k_pct(prior_year)

# Calculate each player's K% from the immediately preceding season.
# This creates the prior-season K% baseline.
prior_rate = player_rate(prior_year, "prior_season_k_pct")

# Calculate each player's K% across all available seasons before the target year.
# This creates the available-history K% baseline.
available_history_rate = player_rate(history, "available_history_k_pct")

# Calculate each player's TBF-weighted K% over the previous two seasons.
# This creates the recent K% baseline.
recent_rate = player_rate(recent_rate_history, "recent_k_pct")

# Create the Marcel table and calculate how many seasons each row is behind the target.
marcel = marcel_history.assign(_lag=target_season - marcel_history["Season"])

# Assign recency weights of 3, 2, and 1, giving newer seasons more influence.
marcel["_weight"] = marcel["_lag"].map(MARCEL_WEIGHTS)

# Combine workload and recency so larger, newer samples receive more influence.
marcel["_weighted_tbf"] = marcel["TBF"] * marcel["_weight"]

# Estimate recency-weighted strikeouts for the Marcel K% calculation.
marcel["_weighted_strikeouts"] = marcel["K%"] * marcel["_weighted_tbf"]
```

```

# Give Stuff+ a workload weight only when that row contains a valid Stuff+ value.
marcel["_stuff_weight"] = marcel["_weighted_tbf"].where(marcel["Stuff+"].notna(), 0.0)

# Weight valid Stuff+ values by workload and recency; missing values contribute nothing.
marcel["_weighted_stuff"] = marcel["Stuff+"].fillna(0.0) * marcel["_stuff_weight"]

# Calculate the League's recency- and TBF-weighted K% to use as Marcel's shrinkage target.
league_marcel_k_pct = marcel["_weighted_strikeouts"].sum() / marcel["_weighted_tbf"].sum()

# Combine each player's weighted strikeouts, TBF, and Stuff+ information across the Marcel
marcel_totals = marcel.groupby("PlayerId").agg(_weighted_strikeouts=("_weighted_strikeouts",
sum), _weighted_tbf=("_weighted_tbf", sum), _weighted_stuff=("_weighted_stuff", sum))

# MARCEL PLAYER-LEVEL RESULTS
# Blend each player's weighted history with imaginary TBF at the League-average K%.
# More regression TBF creates a stronger pull toward the League average.
marcel_totals["marcel_k_pct"] = (marcel_totals["_weighted_strikeouts"] + regression_tbf *
marcel_totals["_weighted_tbf"]) / (marcel_totals["_weighted_tbf"] + regression_tbf)

# Measure how much of the Marcel calculation comes from the player's own history.
# More player history produces a value closer to 1 and indicates greater reliability.
marcel_totals["marcel_reliability"] = marcel_totals["_weighted_tbf"] / (marcel_totals["_weighted_tbf"] +
regression_tbf)

# Divide each player's weighted Stuff+ total by its valid weight to produce their weighted
marcel_totals["weighted_stuff_plus"] = marcel_totals["_weighted_stuff"] / marcel_totals["_stuff_weight"]

# Record which season each row is predicting.
targets["target_season"] = target_season

# [FEATURE 1/22 - AGE] Express age as distance from 30 so age effects are easier to interpret.
targets["age_centered"] = targets["Age"] - 30

# [FEATURE 2/22 - AGE] Allow the model to capture a curved age pattern rather than assuming linear.
targets["age_centered_squared"] = targets["age_centered"] ** 2

# [BASELINE 1/4] Match each target player to their immediately prior-season K%.
targets["prior_season_k_pct"] = targets["PlayerId"].map(prior_rate)

# [FEATURE 3/22 - K%] Match each player to their latest available K%, even if they missed targets.
targets["most_recent_k_pct"] = targets["PlayerId"].map(latest["K%"])

# [BASELINE 2/4] Match each player to their TBF-weighted K% across all available history.
targets["available_history_k_pct"] = targets["PlayerId"].map(available_history_rate)

# [BASELINE 3/4] Match each player to their TBF-weighted K% from the previous two seasons.
targets["recent_k_pct"] = targets["PlayerId"].map(recent_rate)

# [BASELINE 4/4] Match each player to their recency-, workload-, and League-adjusted Marcel K%.
targets["marcel_k_pct"] = targets["PlayerId"].map(marcel_totals["marcel_k_pct"])

# [FEATURE 4/22 - RELIABILITY] Record how much of each Marcel projection comes from the player's own history.
targets["marcel_reliability"] = targets["PlayerId"].map(marcel_totals["marcel_reliability"])

```

```

# [FEATURE 5/22 - WORKLOAD] Record each player's TBF from the immediately prior season.
targets["prior_tbf"] = targets["PlayerId"].map(prior_year.groupby("PlayerId")["TBF"].sum())

# [FEATURE 6/22 - WORKLOAD] Record each player's total TBF from the previous two seasons.
targets["recent_tbf"] = targets["PlayerId"].map(recent_rate_history.groupby("PlayerId")["TBF"].sum())

# [FEATURE 7/22 - WORKLOAD] Record each player's total TBF across all available history.
targets["available_history_tbf"] = targets["PlayerId"].map(history.groupby("PlayerId")["TBF"].sum())

# [FEATURE 8/22 - RECENCY] Record the most recent season in which each player has data.
targets["latest_observed_season"] = targets["PlayerId"].map(latest["Season"])

# [FEATURE 9/22 - RECENCY] Measure how long each player has been absent from the data.
targets["years_since_last_observed"] = target_season - targets["latest_observed_season"]

# Calculate each player's TBF-weighted Stuff+ from the immediately prior season.
prior_stuff = player_weighted_mean(prior_year, "Stuff+", "prior_stuff_plus")

# Calculate each player's TBF-weighted Stuff+ from two seasons earlier.
prior2_stuff = player_weighted_mean(two_years_ago, "Stuff+", "prior2_stuff_plus")

# [FEATURE 10/22 - STUFF+] Match each player to their prior-season weighted Stuff+.
targets["prior_stuff_plus"] = targets["PlayerId"].map(prior_stuff)

# [FEATURE 11/22 - STUFF+] Match each player to their most recently observed Stuff+.
targets["most_recent_stuff_plus"] = targets["PlayerId"].map(latest["Stuff+"])

# [FEATURE 12/22 - STUFF+] Match each player to their three-year recency- and TBF-weighted Stuff+.
targets["weighted_stuff_plus"] = targets["PlayerId"].map(marcel_totals["weighted_stuff_plus"])

# [FEATURE 13/22 - STUFF+] Measure whether each player's Stuff+ improved or declined from prior season.
# reindex puts both calculations in the same player order before subtraction.
targets["stuff_plus_change"] = prior_stuff.reindex(targets["PlayerId"]).to_numpy() - prior2_stuff.reindex(targets["PlayerId"]).to_numpy()

# [FEATURE 14/22 - MISSINGNESS] Create a 1/0 flag showing whether the player has no record in the prior season.
# Missing history may signal MLB entry, injury, or time outside the dataset.
targets["no_observed_history"] = targets["available_history_tbf"].isna().astype(int)

# [FEATURE 15/22 - MISSINGNESS] Create a 1/0 flag showing whether the player missed the immediately prior season.
targets["missed_prior_season"] = targets["prior_tbf"].isna().astype(int)

# [FEATURE 16/22 - MISSINGNESS] Create a 1/0 flag showing whether prior-season Stuff+ is missing.
targets["missing_prior_stuff_plus"] = targets["prior_stuff_plus"].isna().astype(int)

# [FEATURE 17/22 - MISSINGNESS] Create a 1/0 flag showing whether the player's Stuff+ change is missing.
targets["missing_stuff_plus_change"] = targets["stuff_plus_change"].isna().astype(int)

# List the K% features that require a neutral replacement when player history is missing.
rate_columns = ["prior_season_k_pct", "most_recent_k_pct", "available_history_k_pct", "recent_rate_k_pct"]

# Replace missing player K% values with the prior-season League K%.

```

```

# This gives players without history a reasonable starting prediction.
targets[rate_columns] = targets[rate_columns].fillna(league_prior_k_pct)

# Replace missing workload with zero because missing history means no recorded TBF.
targets[["prior_tbf", "recent_tbf", "available_history_tbf"]] = targets[["prior_tbf", "recent_tbf", "available_history_tbf"]].fillna(0)

# [FEATURE 18/22 - WORKLOAD] Compress prior-season workload so extremely large TBF values
# log1p means log(1 + TBF), which also allows a workload of zero.
targets["log1p_prior_tbf"] = np.log1p(targets["prior_tbf"])

# [FEATURE 19/22 - WORKLOAD] Compress recent workload while preserving the difference between prior and recent workload.
targets["log1p_recent_tbf"] = np.log1p(targets["recent_tbf"])

# [FEATURE 20/22 - WORKLOAD] Compress total historical workload so a few very large career
# log1p means log(1 + TBF), which also allows a workload of zero.
targets["log1p_available_history_tbf"] = np.log1p(targets["available_history_tbf"])

# Assign zero Marcel reliability when a player has no history supporting their projection.
targets["marcel_reliability"] = targets["marcel_reliability"].fillna(0.0)

# [FEATURE 21/22 - LEAGUE CONTEXT] Give every player the immediately prior season's League K% used in Marcel.
targets["league_prior_k_pct"] = league_prior_k_pct

# [FEATURE 22/22 - LEAGUE CONTEXT] Give every player the recency-weighted League K% used in Marcel.
targets["league_recent_k_pct"] = league_marcel_k_pct

# Return the completed player-level table of baselines and candidate model features.
return targets

```

```

# Define a reusable function that tests one Marcel regression-TBF value across all backtest seasons.
def build_backtest(regression_tbf):

    # Create an empty list that will store each season's prediction results.
    folds = []

    # Process each historical target season separately.
    for season in BACKTEST_SEASONS:

        # Select the players and actual results from the season being predicted.
        # K% and TBF are retained only for evaluating the completed predictions.
        actual = development_df.loc[development_df["Season"] == season, ["PlayerId", "Name", "K%", "TBF"]]

        # Build predictions using only seasons before the current target season.
        predictions = build_baselines(development_df, actual, season, regression_tbf)

        # Match each player's predictions with their actual K% and TBF.
        # Store the completed season-level backtest table in the folds list.
        folds.append(predictions.merge(actual[["PlayerId", "K%", "TBF"]], on="PlayerId"))

    # Combine all season-level backtests into one table.
    # ignore_index creates a clean row index across the combined seasons.
    return pd.concat(folds, ignore_index=True)

# Test shrinkage using 2022-2023 only so 2024 is not used to make modeling decisions.
# Create an empty list to store the result for each candidate regression-TBF value.
tuning_rows = []

```

```

# Try every possible league-average shrinkage strength in the predefined grid.
for regression_tbf in REGRESSION_TBF_GRID:

    # Build historical predictions using the current regression-TBF value.
    candidate = build_backtest(regression_tbf)

    # Keep only 2022-2023 predictions for tuning.
    # .le(2023) means "less than or equal to 2023."
    tuning_sample = candidate.loc[candidate["target_season"].le(2023)]

    # Calculate each player's absolute Marcel prediction error.
    absolute_error = (tuning_sample["marcel_k_pct"] - tuning_sample["K%"]).abs()

    # Store the candidate value and its two error measurements.
    # Regular MAE treats every player equally.
    # TBF-weighted MAE gives higher-workload players more influence.
    tuning_rows.append({"regression_tbf": regression_tbf, "mae": absolute_error.mean(), "tbf_w": absolute_error.mean() * regression_tbf})

# Convert the results into a table and rank candidates from lowest to highest MAE.
# Regular MAE is the metric used to select the winner.
marcel_tuning = pd.DataFrame(tuning_rows).sort_values("mae").reset_index(drop=True)

# Select the regression-TBF value from the first row, which has the lowest MAE.
BEST_REGRESSION_TBF = int(marcel_tuning.loc[0, "regression_tbf"])

# Display the candidate results rounded to four decimal places.
display(marcel_tuning.round(4))

# Print the selected number of imaginary league-average TBF.
print(f"Selected Marcel regression strength: {BEST_REGRESSION_TBF} pseudo-TBF")

```

	regression_tbf	mae	tbf_weighted_mae
0	400	0.0600	0.0351
1	200	0.0600	0.0355
2	100	0.0603	0.0364
3	800	0.0606	0.0355
4	50	0.0608	0.0373

Selected Marcel regression strength: 400 pseudo-TBF

```

# Rebuild the historical baseline predictions using the selected Marcel shrinkage strength.
baseline_backtest = build_backtest(BEST_REGRESSION_TBF)

# List the four simple K% projections that will be compared.
baseline_columns = ["prior_season_k_pct", "available_history_k_pct", "recent_k_pct", "marcel_k_pct"]

# Create an empty list to store each baseline's evaluation results.
metric_rows = []

# Evaluate each baseline using the same historical player-seasons.
for column in baseline_columns:
    metric_rows.append(baseline_backtest[column].mean())

```

```

# Calculate signed error: predicted K% minus actual K%.
# Positive values are overpredictions; negative values are underpredictions.
error = baseline_backtest[column] - baseline_backtest["K%"]

# Store three measures of prediction accuracy for the current baseline.
# MAE measures the average absolute miss.
# RMSE places more emphasis on especially large misses.
# TBF-weighted MAE gives higher-workload target seasons more influence.
metric_rows.append({"projection": column, "mae": error.abs().mean(), "rmse": np.sqrt(np.me

# Convert the results into a table and rank the baselines from lowest to highest MAE.
baseline_metrics = pd.DataFrame(metric_rows).sort_values("mae").reset_index(drop=True)

# Clearly Label and display the baseline accuracy comparison.
print("BASELINE PERFORMANCE COMPARISON")
display(baseline_metrics.round(4))

# Create 2025 baseline projections and candidate features using data through 2024.
# Actual 2025 performance is not included.
features_2025 = build_baselines(development_df, players_2025, 2025, BEST_REGRESSION_TBF)

# Allow pandas to display every column.
pd.set_option("display.max_columns", None)

# Clearly Label and preview historical rows containing predictions and actual outcomes.
print("HISTORICAL BACKTEST TABLE – FEATURES, PREDICTIONS, AND ACTUAL RESULTS")
display(baseline_backtest.head())

# Clearly Label and preview 2025 rows containing features but no actual 2025 K%.
print("2025 FEATURE TABLE – BASELINES AND CANDIDATE MODEL FEATURES")
display(features_2025.head())

```

BASELINE PERFORMANCE COMPARISON

	projection	mae	rmse	tbf_weighted_mae
0	marcel_k_pct	0.0595	0.0853	0.0345
1	available_history_k_pct	0.0612	0.0891	0.0391
2	recent_k_pct	0.0614	0.0895	0.0395
3	prior_season_k_pct	0.0631	0.0912	0.0405

HISTORICAL BACKTEST TABLE – FEATURES, PREDICTIONS, AND ACTUAL RESULTS

	PlayerId	Name	Age	target_season	age_centered	age_centered_squared	prior_season_k_pct	m
0	593160	Whit Merrifield	33	2022	3	9	0.231799	
1	592743	Andrelton Simmons	32	2022	2	4	0.231799	
2	624512	Reese McGuire	27	2022	-3	9	0.231799	
3	405395	Albert Pujols	42	2022	12	144	0.231799	
4	643524	Frank Schwindel	30	2022	0	0	0.231799	

2025 FEATURE TABLE — BASELINES AND CANDIDATE MODEL FEATURES

	PlayerId	Name	Age	target_season	age_centered	age_centered_squared	prior_season_k_pct	m
0	608671	Travis Jankowski	34	2025	4	16	0.2258	
1	643396	Isiah Kiner-Falefa	30	2025	0	0	0.0000	
2	650489	Willi Castro	28	2025	-2	4	0.0000	
3	623168	Tyler Heineman	34	2025	4	16	0.0000	
4	682515	Logan Porter	29	2025	-1	1	0.2258	

Additional candidate features

The feature table now includes workload, availability, pitch quality, age, and league context:

- **Workload:** prior-season, recent two-year, and available-history TBF.
- **Availability:** most recent observed season, years since that season, and indicators for no prior history or a missed prior season.
- **Stuff+:** prior-season, most recent, and 3/2/1 recency- and TBF-weighted Stuff+, plus the change from \$t-2\$ to \$t-1\$.
- **Age:** target-season age is represented by $\text{age_centered} = \text{Age} - 30$ and $\text{age_centered_squared} = (\text{Age} - 30)^2$. Thirty is a fixed reference within the dense part of the observed age distribution; it is not an assumption that performance peaks at age 30. The squared term allows the relationship between age and future K% to curve rather than forcing the effect of every additional year to remain constant.

- **League context:** the prior-season league K% and three-year recency-weighted league K%.

Available-history features use only the seasons present before each target year in the 2021–2024 data window; they are not complete-career measures. Earlier MLB seasons are not present. Unavailable Stuff+ changes remain missing and receive explicit missingness indicators. Any numerical imputation will later be learned inside each training fold rather than from the full dataset.

Feature quality assurance

Before statistical modeling, the code audits every candidate feature in both the historical backtest table and the 2025 projection table.

The audit:

- Confirms each candidate feature's data type.
- Counts missing values in the historical backtest data.
- Counts and calculates the percentage of missing values in the 2025 projection data.
- Confirms that essential K%, workload, age, and league-context features are complete for every 2025 player.
- Confirms that each `PlayerId` appears only once in the 2025 table.
- Confirms that all TBF values are nonnegative.
- Allows expected missing values in optional features such as Stuff+, while preserving missingness indicators that tell the model those values were unavailable.

The `assert` statements stop execution if a required condition fails. Passing these checks confirms that the 2025 feature table has one valid row per player and that its essential inputs are ready for modeling. It does not confirm that every candidate feature will be selected or that the model will predict accurately; those questions are evaluated later through feature selection and historical validation.

```
# List every feature that may be tested as a model input.
# These are candidates; the final model may use only a selected subset.
candidate_feature_columns = [
    # Age features test whether expected K% changes differently across a pitcher's career.
    "age_centered", "age_centered_squared",

    # Historical K% features describe the pitcher's previous strikeout performance.
    "prior_season_k_pct", "most_recent_k_pct", "available_history_k_pct",
    "recent_k_pct", "marcel_k_pct", "marcel_reliability",

    # Workload features describe how much evidence supports the historical rates.
    "prior_tbf", "recent_tbf", "available_history_tbf",
    "log1p_prior_tbf", "log1p_recent_tbf", "log1p_available_history_tbf",

    # Recency features identify how recently the pitcher appeared in the data.
    "latest_observed_season", "years_since_last_observed",

    # Stuff+ features describe pitch quality and whether it recently changed.
    "prior_stuff_plus", "most_recent_stuff_plus",
    "weighted_stuff_plus", "stuff_plus_change",
```

```

# League features provide the strikeouts environment surrounding each prediction.
"league_prior_k_pct", "league_recent_k_pct",

# Missingness flags tell the model which historical information was unavailable.
"no_observed_history", "missed_prior_season",
"missing_prior_stuff_plus", "missing_stuff_plus_change",
]

# Create a table that checks each candidate feature's data type and missing values.
feature_audit = pd.DataFrame({

    # Confirm that each feature has the expected numeric data type.
    "dtype": features_2025[candidate_feature_columns].dtypes.astype(str),

    # Count missing values in the historical backtest data.
    "backtest_missing": baseline_backtest[candidate_feature_columns].isna().sum(),

    # Count missing values in the 2025 projection data.
    "projection_missing": features_2025[candidate_feature_columns].isna().sum(),
})

# Calculate the percentage of 2025 players missing each feature.
# A percentage is easier to interpret than a count when comparing columns.
feature_audit["projection_missing_pct"] = feature_audit["projection_missing"] / len(features_2025)

# Display the audit so missing-data problems can be reviewed before modeling.
display(feature_audit)

# List the essential features that must contain values for every 2025 player.
# Optional Stuff+ and recency fields are excluded because they may be legitimately missing.
core_complete = [
    "Age", "age_centered", "age_centered_squared",
    "prior_season_k_pct", "most_recent_k_pct",
    "available_history_k_pct", "recent_k_pct", "marcel_k_pct",
    "prior_tbf", "recent_tbf", "available_history_tbf",
    "log1p_prior_tbf", "log1p_recent_tbf", "log1p_available_history_tbf",
    "league_prior_k_pct", "league_recent_k_pct",
]

# Stop the notebook if any essential 2025 feature is missing.
assert features_2025[core_complete].notna().all().all()

# Stop the notebook if a player appears more than once in the 2025 feature table.
assert features_2025["PlayerId"].is_unique

# Stop the notebook if any workload value is negative, because TBF cannot be below zero.
assert features_2025[["prior_tbf", "recent_tbf", "available_history_tbf"]].ge(0).all().all()

# Confirm that the feature-quality checks passed for every projected player.
print(f"Feature audit passed for all {len(features_2025):,} projected players.")

# Show original rows where K% exists but Stuff+ is missing.
missing_stuff_examples = df.loc[df["K%"].notna() & df["Stuff+"].isna(), ["PlayerId", "Name", ""]]

```

```
display(missing_stuff_examples.head(10))
print(f"Rows with K% but missing Stuff+: {len(missing_stuff_examples):,}")
```

	dtype	backtest_missing	projection_missing	projection_missing_pct
age_centered	int64	0	0	0.000000
age_centered_squared	int64	0	0	0.000000
prior_season_k_pct	float64	0	0	0.000000
most_recent_k_pct	float64	0	0	0.000000
available_history_k_pct	float64	0	0	0.000000
recent_k_pct	float64	0	0	0.000000
marcel_k_pct	float64	0	0	0.000000
marcel_reliability	float64	0	0	0.000000
prior_tbf	float64	0	0	0.000000
recent_tbf	float64	0	0	0.000000
available_history_tbf	float64	0	0	0.000000
log1p_prior_tbf	float64	0	0	0.000000
log1p_recent_tbf	float64	0	0	0.000000
log1p_available_history_tbf	float64	0	0	0.000000
latest_observed_season	float64	660	174	0.199313
years_since_last_observed	float64	660	174	0.199313
prior_stuff_plus	float64	762	258	0.295533
most_recent_stuff_plus	float64	660	174	0.199313
weighted_stuff_plus	float64	660	184	0.210767
stuff_plus_change	float64	1691	421	0.482245
league_prior_k_pct	float64	0	0	0.000000
league_recent_k_pct	float64	0	0	0.000000
no_observed_history	int64	0	0	0.000000
missed_prior_season	int64	0	0	0.000000
missing_prior_stuff_plus	int64	0	0	0.000000
missing_stuff_plus_change	int64	0	0	0.000000

Feature audit passed for all 873 projected players.

PlayerId Name Season K% TBF Stuff+

Rows with K% but missing Stuff+: 0

Exploratory distribution and outlier audit

This audit uses only 2021–2024 performance data and serves three purposes:

1. **Validate the inputs:** confirm that ranges, missingness, and season-level patterns are plausible and identify possible data errors.
2. **Choose appropriate handling:** determine where the model may need sample-size weighting, regression to the mean, transformations, imputation, or missingness indicators.
3. **Document exclusion decisions:** make any removal rule explicit rather than silently discarding inconvenient observations.

The audit is not intended to make the data look more regular. A flag identifies an observation for review; it is not an automatic exclusion rule. Unusual values may be legitimate, particularly for position players pitching or pitchers with very few TBF. A row would be removed only if there were strong evidence of a data error or that it fell outside the population the project asks us to predict.

```
# List the important raw variables to inspect before modeling.
audit_variables = ["K%", "TBF", "Stuff+", "Age"]

# Select percentiles that show the center, spread, and extreme ends of each distribution.
# The 1st and 99th percentiles help identify unusually low or high values.
audit_percentiles = [0.01, 0.05, 0.25, 0.50, 0.75, 0.95, 0.99]

# Summarize each variable's count, mean, standard deviation, percentiles, minimum, and maximum
distribution_summary = development_df[audit_variables].describe(percentiles=audit_percentiles)

# Display variables as rows and round the statistics for easier reading.
display(distribution_summary.round(4))

# Estimate strikeouts for each row because the dataset provides K% and TBF rather than strikeouts
development_df_with_strikeouts = development_df.assign(_estimated_strikeouts=development_df["K%"] * development_df["TBF"])

# Group the development data by season to measure each season's overall league environment.
league_environment = development_df_with_strikeouts.groupby("Season").agg(player_seasons=("Pl",

# Divide total estimated strikeouts by total TBF to calculate a properly workload-weighted league environment
league_environment["league_k_pct"] = league_environment["estimated_strikeouts"] / league_environment["TBF"]

# Display the league-level results for each season.
display(league_environment.round(4))
```

	count	mean	std	min	1%	5%	25%	50%	75%	95%
K%	3498.0	0.2051	0.0901	0.0000	0.0000	0.0000	0.1636	0.2097	0.2558	0.3333
TBF	3498.0	208.8113	204.8480	1.0000	3.0000	6.0000	47.0000	154.0000	278.0000	687.3000
Stuff+	3498.0	95.4315	17.9204	-46.5108	23.4528	54.1992	90.8664	97.9113	104.7547	115.6876
Age	3498.0	28.4537	3.5815	20.0000	22.0000	24.0000	26.0000	28.0000	31.0000	35.0000



	player_seasons	total_tbf	estimated_strikeouts	league_k_pct
Season				
2021	909	181817	42145.0000	0.2318
2022	871	182052	40812.0000	0.2242
2023	863	184104	41843.0000	0.2273
2024	855	182449	41196.9999	0.2258

```
# Import Matplotlib's plotting tools and use the standard abbreviation "plt."
import matplotlib.pyplot as plt

# Create one figure containing four charts arranged in two rows and two columns.
# figsize controls the overall chart size in inches.
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Define the data, chart title, and horizontal-axis label for each histogram.
plot_specs = [
    # Convert decimal K% values to percentage points for easier interpretation.
    (development_df["K%"] * 100, "K%", "Percent"),

    # Display the distribution of player-season workloads.
    (development_df["TBF"], "TBF", "Batters faced"),

    # Display the distribution of pitch-quality ratings.
    (development_df["Stuff+"], "Stuff+", "Stuff+"),

    # Display the distribution of player ages.
    (development_df["Age"], "Age", "Years"),
]

# Match each chart location with one set of plotting instructions.
# axes.flat turns the 2x2 chart grid into a simple sequence of four axes.
for axis, (values, title, x_label) in zip(axes.flat, plot_specs):

    # Remove missing values because they cannot be placed into histogram bins.
    clean_values = values.dropna()

    # Treat age differently because it is measured in whole years.
    if title == "Age":

        # Center one histogram bar on each whole-number age.
```

```

    histogram_bins = np.arange(clean_values.min() - 0.5, clean_values.max() + 1.5, 1)

    # Label every second age to keep the horizontal axis readable.
    axis.set_xticks(np.arange(clean_values.min(), clean_values.max() + 1, 2))

    # Use 30 equal-width bins for the continuous K%, TBF, and Stuff+ variables.
    else:
        histogram_bins = 30

    # Draw the histogram using the selected bins and consistent colors.
    axis.hist(clean_values, bins=histogram_bins, color="#8B1E3F", edgecolor="white")

    # Add a dashed line at the median to show the center of the distribution.
    axis.axvline(values.median(), color="#1F4E79", linestyle="--", label="Median")

    # Add a descriptive title to the individual chart.
    axis.set_title(f"Distribution of {title}")

    # Label the horizontal axis with the variable's unit.
    axis.set_xlabel(x_label)

    # Show that bar heights count player-season observations.
    axis.set_ylabel("Player-seasons")

    # Display the label identifying the median line.
    axis.legend()

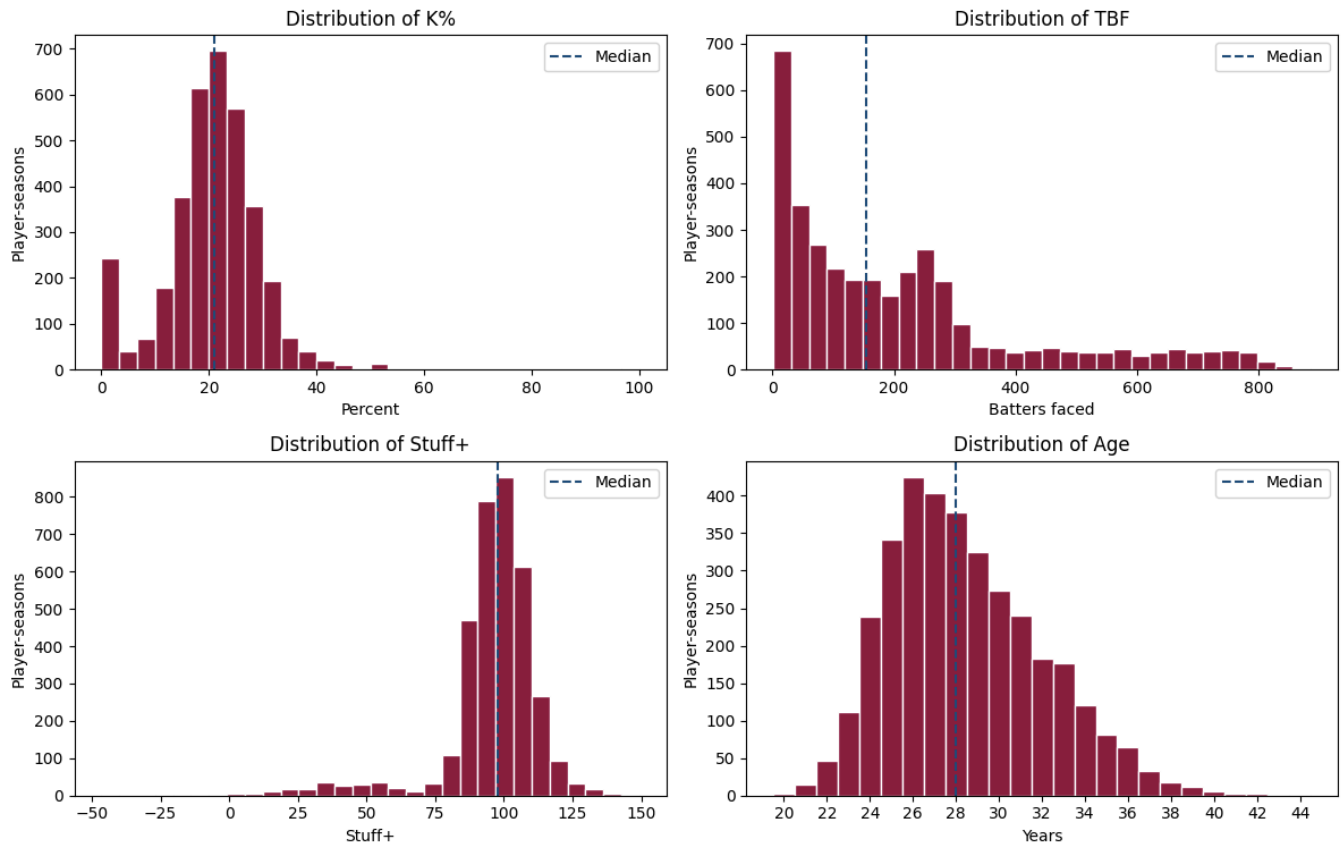
# Add one overall title above all four charts.
fig.suptitle("2021-2024 Development Data Distributions", fontsize=14)

# Adjust spacing so titles and labels do not overlap.
fig.tight_layout()

# Render the completed figure in the notebook.
plt.show()

```

2021-2024 Development Data Distributions



```
# Set the Lower Stuff+ review threshold at the bottom 1% of observed values.
STUFF_LOW = development_df["Stuff+"].quantile(0.01)

# Set the upper Stuff+ review threshold at the top 1% of observed values.
STUFF_HIGH = development_df["Stuff+"].quantile(0.99)

# Copy the development data so review flags can be added without changing the original table.
review_flags = development_df.copy()

# Flag samples below 50 TBF because their K% may be especially unstable.
review_flags["small_sample"] = review_flags["TBF"] < 50

# Flag K% below 5% or above 45% as unusual values requiring review.
review_flags["extreme_k_pct"] = (review_flags["K%"] < 0.05) | (review_flags["K%"] > 0.45)

# Flag Stuff+ values in the bottom or top 1% of the observed distribution.
review_flags["extreme_stuff_plus"] = (review_flags["Stuff+"] < STUFF_LOW) | (review_flags["Stuff+"] > STUFF_HIGH)

# Flag ages outside the expected professional-player range for closer review.
review_flags["unusual_age"] = (review_flags["Age"] < 20) | (review_flags["Age"] > 45)

# List the review flags that will be summarized together.
flag_columns = ["small_sample", "extreme_k_pct", "extreme_stuff_plus", "unusual_age"]

# Count and calculate the proportion of rows receiving each review flag.
# Boolean True values behave like 1, while False values behave like 0.
```

```

flag_summary = pd.DataFrame({"flagged_rows": review_flags[flag_columns].sum(), "flagged_pct":

# Display the number and proportion of flagged records.
display(flag_summary.round(4))

# Count all rows with an extreme K%.
extreme_k_count = int(review_flags["extreme_k_pct"].sum())

# Count extreme K% rows that also have fewer than 50 TBF.
# This checks whether unusual K% values are concentrated in unreliable small samples.
extreme_k_small = int((review_flags["extreme_k_pct"] & review_flags["small_sample"]).sum())

# Measure the difference between the highest and lowest annual League K%.
league_k_range = league_environment["league_k_pct"].max() - league_environment["league_k_pct"]

# Calculate the percentage of extreme K% rows that have fewer than 50 TBF.
extreme_k_small_pct = extreme_k_small / extreme_k_count

# Display the result as a percentage with one decimal place.
print(f"{extreme_k_small} of {extreme_k_count} extreme K% rows have fewer than 50 TBF ({extreme_k_small_pct:.1f}%)")

# Report how much the overall strikeout environment changed across seasons.
print(f"League K% varied by {league_k_range * 100:.2f} percentage points from 2021-2024.")

# Report the data-based Stuff+ thresholds used to flag the bottom and top 1%.
print(f"Stuff+ review thresholds: below {STUFF_LOW:.1f} or above {STUFF_HIGH:.1f}." )

# Flag rows meeting at least one review condition.
# axis=1 checks across the flag columns within each individual row.
review_flags["any_review_flag"] = review_flags[flag_columns].any(axis=1)

# List the values and flags needed to review unusual records.
review_columns = ["Season", "Name", "PlayerId", "K%", "TBF", "Stuff+", "Age", *flag_columns]

# Select and sort only rows with extreme K%.
extreme_k_rows = review_flags.loc[review_flags["extreme_k_pct"], review_columns].sort_values(["K%", "TBF"])

# Place the first 20 and final 20 extreme K% rows beside each other.
extreme_k_comparison = pd.concat({"First 20": extreme_k_rows.head(20).reset_index(drop=True), "Last 20": extreme_k_rows.tail(20).reset_index(drop=True)})

# Display the lowest- and highest-workload extreme K% rows together.
display(extreme_k_comparison)

```

	flagged_rows	flagged_pct
small_sample	924	0.2642
extreme_k_pct	280	0.0800
extreme_stuff_plus	70	0.0200
unusual_age	0	0.0000

276 of 280 extreme K% rows have fewer than 50 TBF (98.6%).
League K% varied by 0.76 percentage points from 2021-2024.
Stuff+ review thresholds: below 23.5 or above 126.2.

	Season	Name	PlayerId	K%	TBF	Stuff+	Age	small_sample	extreme_k_pct	extreme_stu
0	2021	Jack Mayfield	608686	0.0	1	44.793333	30	True	True	
1	2021	Kevin Pillar	607680	0.0	1	52.500000	32	True	True	
2	2021	Andrew Knapp	595284	0.0	1	59.540000	29	True	True	
3	2021	Hanser Alberto	593643	0.0	1	76.800000	28	True	True	
4	2021	Matt Duffy	622110	0.0	1	81.688000	30	True	True	
5	2021	Sergio Alcántara	642727	0.0	1	90.335000	24	True	True	
6	2022	Marwin Gonzalez	503556	0.0	1	47.390000	33	True	True	
7	2023	Hagen Danner	668470	0.0	1	96.062857	24	True	True	
8	2024	Luis Torrens	620443	0.0	1	19.260000	28	True	True	
9	2024	Jhonny Pereda	640902	0.0	1	54.460000	28	True	True	
10	2024	Miguel Sanó	593934	0.0	1	61.420000	31	True	True	
11	2024	Wander Suero	593833	0.0	1	115.900000	32	True	True	
12	2021	Mike Freeman	502273	1.0	1	40.400000	33	True	True	
13	2021	Romy Gonzalez	663853	1.0	1	58.050000	24	True	True	
14	2021	Nick Maton	665155	1.0	1	69.660000	24	True	True	
15	2021	Austin Slater	596103	0.0	2	68.446667	28	True	True	
16	2022	Darick Hall	669742	0.0	2	72.180000	26	True	True	
17	2023	Seby Zavala	664874	0.0	2	89.320000	29	True	True	
18	2024	Evan Kravetz	667478	0.0	2	85.995000	27	True	True	

	Season	Name	PlayerId	K%	TBF	Stuff+	Age	small_sample	extreme_k_pct	extreme_stu
19	2023	Chris Vallimont	681808	0.5	2	94.525455	26	True	True	

Audit findings and modeling implications

- **Small samples are common:** 924 of 3,498 development player-seasons (26.4%) contain fewer than 50 TBF. This supports weighting observations by opportunity and shrinking unstable rates toward the league mean.
- **Extreme K% is primarily a sample-size issue:** 276 of the 280 observations below 5% or above 45% K% have fewer than 50 TBF. These values are retained because the projection must cover every listed player; TBF and shrinkage will limit their influence.
- **The league strikeout environment was fairly stable:** TBF-weighted MLB K% ranged from 22.42% to 23.18% during 2021–2024, a difference of 0.76 percentage points. League context remains available as a feature, but large seasonal adjustments are unlikely to be necessary.
- **Stuff+ has meaningful tails:** the diagnostic 1st- and 99th-percentile thresholds are 23.5 and 126.2, flagging 70 observations. Reviewed examples include position players pitching and extremely small samples, so these records are flagged rather than automatically removed. These percentile thresholds identify observations for review; they are not baseball-validity cutoffs.
- **Age values are plausible:** ages range from 20 to 44, with no observations outside the review range of 20–45. Centered age and centered age squared will allow the model to test for a nonlinear age effect.

Row-retention decision: no observations are removed. The review found unusual but plausible baseball records rather than clear data errors. Removing small-sample pitchers would also make the training population less representative of the 873 players who must receive predictions. Their uncertainty will instead be handled through TBF-aware features and regression to the mean.

Feature decision: audit flags do not determine which predictors enter the final model. Missingness indicators are preserved, numerical imputation will be fit only within each training fold, and feature groups will be retained only when they improve historical out-of-sample accuracy. No 2025 performance data were used in this audit.

Fitted model development using time-ordered validation

The earlier section backtested four simple K% projections and established Marcel as the strongest non-fitted benchmark. This section tests whether OLS, Ridge, or histogram gradient boosting can improve upon Marcel by combining the candidate features.

Before fitted-model testing, the 26 audited candidate columns are reduced to 22 model inputs. The three raw TBF columns are replaced by their log-transformed versions to reduce workload skew and represent diminishing gains in reliability. `latest_observed_season` is replaced by `years_since_last_observed`, which more directly measures how old the player's history is. These four exclusions are deliberate feature-design decisions; the remaining feature groups are then evaluated using time-ordered validation.

Because the data begin in 2021, the fitted-model timeline is:

- **2022 training data:** Build predictors from 2021 history and use actual 2022 K% as the outcome.
- **2023 development forecast:** Train on 2022 and predict 2023. Use this result to select the Ridge penalty and feature group. Prefer fewer features when MAE differs by no more than 0.01 percentage points.
- **2024 final validation:** Refit the finalized models on 2022–2023 and predict the untouched 2024 season.
- **2025 projection:** After validation, refit the selected model using all available labeled player-seasons and predict 2025.

The fitted approaches serve different purposes:

- **OLS:** provides an interpretable linear comparison but may be unstable with overlapping predictors.
- **Ridge:** adds a coefficient penalty to stabilize correlated features and reduce overfitting.
- **Histogram gradient boosting:** tests whether nonlinear relationships and interactions justify additional complexity.

Random row splits are avoided because they would mix seasons and fail to reproduce the real forecasting task. MAE is the primary metric, while RMSE and TBF-weighted MAE provide additional views of large errors and high-workload players.

Documentation: https://scikit-learn.org/stable/modules/linear_model.html and <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html>

```
# Import OrderedDict so feature sets remain in the intended testing order.
from collections import OrderedDict

# Import a nonlinear tree-based model to test whether complexity improves predictions.
from sklearn.ensemble import HistGradientBoostingRegressor

# Import a tool that fills missing feature values using training-data information.
from sklearn.impute import SimpleImputer

# Import OLS as a basic linear comparison and Ridge as a regularized linear model.
from sklearn.linear_model import LinearRegression, Ridge

# Import Pipeline so imputation, scaling, and modeling occur in the correct order.
from sklearn.pipeline import Pipeline

# Import StandardScaler so Ridge compares features measured on different scales fairly.
from sklearn.preprocessing import StandardScaler
```

```

# Group historical K% features that describe the player's previous results.
HISTORY_FEATURES = [
    "prior_season_k_pct", "most_recent_k_pct", "available_history_k_pct",
    "recent_k_pct", "marcel_k_pct", "marcel_reliability",
]

# Group workload and availability features that describe how much history supports those results.
# Log TBF is used instead of raw TBF to capture diminishing gains from additional workload.
WORKLOAD_FEATURES = [
    "log1p_prior_tbf", "log1p_recent_tbf", "log1p_available_history_tbf",
    "years_since_last_observed", "no_observed_history", "missed_prior_season",
]

# Group pitch-quality features and indicators showing when Stuff+ information is unavailable.
STUFF_FEATURES = [
    "prior_stuff_plus", "most_recent_stuff_plus",
    "weighted_stuff_plus", "stuff_plus_change",
    "missing_prior_stuff_plus", "missing_stuff_plus_change",
]

# Group the two age features that allow a curved relationship between age and future K%.
AGE_FEATURES = ["age_centered", "age_centered_squared"]

# Group League features that describe the recent strikeout environment.
LEAGUE_FEATURES = ["league_prior_k_pct", "league_recent_k_pct"]

# Define a reusable function that combines multiple feature groups into one ordered list.
# *groups allows the function to receive any number of feature-group lists.
def combine_features(*groups):
    # Combine every group, preserve the original order, and remove duplicate feature names.
    return list(dict.fromkeys(feature for group in groups for feature in group))

# Create progressively larger feature sets so each group's added value can be tested.
FEATURE_SETS = OrderedDict({

    # Test historical K% information by itself.
    "K history": HISTORY_FEATURES,

    # Test whether workload and availability improve upon K% history.
    "+ workload": combine_features(HISTORY_FEATURES, WORKLOAD_FEATURES),

    # Test whether Stuff+ adds useful pitch-quality information.
    "+ Stuff+": combine_features(HISTORY_FEATURES, WORKLOAD_FEATURES, STUFF_FEATURES),

    # Test whether age adds information about expected future change.
    "+ age": combine_features(HISTORY_FEATURES, WORKLOAD_FEATURES, STUFF_FEATURES, AGE_FEATURES),

    # Test whether changes in the League strikeout environment add further value.
    "+ league context": combine_features(HISTORY_FEATURES, WORKLOAD_FEATURES, STUFF_FEATURES,
})

```

```
# Test Ridge penalties from very weak to very strong regularization.
# The broad tenfold steps efficiently show how much coefficient shrinkage is useful.
# alpha controls how strongly Ridge shrinks the feature coefficients.
```

```
RIDGE_ALPHAS = [0.1, 1.0, 10.0, 100.0, 1000.0]
```

```
# Use 2023 for development and 2024 for final out-of-time validation.
```

```
MODEL_VALIDATION_SEASONS = [2023, 2024]
```

```
# Confirm the number of features in the largest fitted-model specification.
```

```
print(f"Largest candidate model: {len(FEATURE_SETS['+ league context'])} features")
```

Largest candidate model: 22 features

How features enter each model

A **training fold** is the set of historical target player-seasons used to teach the model before it predicts a later validation season. For the 2023 forecast, the training fold contains 2022 target player-seasons; for the 2024 forecast, it contains 2022–2023 target player-seasons; and the final 2025 model is refit on 2022–2024 target player-seasons. This term describes the rows used to estimate the model and preprocessing rules, not the earlier seasons summarized inside an individual pitcher's features. For example, a 2023 player's feature row can summarize his 2021–2022 history, while the model itself is trained on the available 2022 target rows from all players.

For each validation season, preprocessing is learned from the earlier training seasons only:

1. Missing continuous values, primarily Stuff+ history, are replaced with the training-fold median. Existing missingness indicators preserve the fact that the value was unavailable.
2. OLS and Ridge features are centered and scaled using training-fold means and standard deviations. This gives the Ridge penalty a comparable meaning across percentage, age, and workload variables.
3. TBF enters through $\log(1 + \text{TBF})$ because the workload distribution is strongly right-skewed. Raw TBF remains available for auditing but is not used alongside its transformed version.
4. Predictions are constrained to the logical 0%–100% range. The notebook records whether clipping was needed.

A pipeline keeps imputation and scaling inside the training fold, preventing information from the validation season from influencing preprocessing.

Documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html> and <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

```
# Define a reusable function that creates a complete workflow for OLS or Ridge.
```

```
# "estimator" represents whichever linear model is passed into the function.
```

```
def make_linear_pipeline(estimator):
```

```
    # Apply each preparation step in order before fitting the model.
```

```
    return Pipeline([
```

```
        # Replace missing feature values with the training data's median.
```

```

# Learning the median inside the pipeline prevents validation-data leakage.
# Retain an all-missing fold column so every fold keeps the same feature schema.
("imputer", SimpleImputer(strategy="median", keep_empty_features=True)),

# Put all features on comparable scales so Ridge penalizes them fairly.
("scaler", StandardScaler()),

# Fit the supplied OLS or Ridge model using the prepared features.
("model", estimator),
])

```

Define a reusable function that creates the histogram gradient-boosting workflow.

```
def make_boosting_pipeline():
```

```

# Apply missing-value handling before fitting the nonlinear model.
return Pipeline([

    # Replace missing values with medians learned only from the training data.
    # Retain an all-missing fold column so every fold keeps the same feature schema.
    ("imputer", SimpleImputer(strategy="median", keep_empty_features=True)),

    # Fit a deliberately restrained boosting model to reduce overfitting.
    ("model", HistGradientBoostingRegressor(

        # Make each new tree contribute a small correction.
        learning_rate=0.05,

        # Allow up to 200 rounds of sequential error correction.
        max_iter=200,

        # Limit each tree's complexity to 15 final decision groups.
        max_leaf_nodes=15,

        # Require at least 30 observations in a final group to avoid tiny-sample rules.
        min_samples_leaf=30,

        # Penalize overly strong adjustments to further reduce overfitting.
        l2_regularization=1.0,

        # Fix the random process so repeated runs produce the same result.
        random_state=27,
    )),
])

```

Define a reusable function that trains on earlier seasons and predicts one later season.

```
def predict_validation_season(model, feature_columns, validation_season, model_name):
```

```

# Use only player-seasons before the validation year for training.
train = baseline_backtest.loc[baseline_backtest["target_season"] < validation_season]

# Keep the selected later season completely separate for prediction and evaluation.
validate = baseline_backtest.loc[baseline_backtest["target_season"] == validation_season]

# Learn the relationship between the selected features and actual future K%.

```

```

# Pipeline preparation is learned only from the training rows.
model.fit(train[feature_columns], train["K%"])

# Generate K% predictions for the held-out validation season.
raw_prediction = model.predict(validate[feature_columns])

# Restrict predictions to the valid K% range of 0% through 100%.
prediction = np.clip(raw_prediction, 0.0, 1.0)

# Return one organized evaluation row for every predicted player-season.
return pd.DataFrame({

    # Preserve player identity for later review.
    "PlayerId": validate["PlayerId"].to_numpy(),
    "Name": validate["Name"].to_numpy(),

    # Record the season and model responsible for each prediction.
    "validation_season": validation_season,
    "model": model_name,

    # Store actual and predicted K% together so error can be calculated.
    "actual_k_pct": validate["K%"].to_numpy(),
    "predicted_k_pct": prediction,

    # Preserve target-season TBF for workload-weighted evaluation.
    "TBF": validate["TBF"].to_numpy(),

    # Record whether an impossible raw prediction required clipping.
    "prediction_was_clipped": raw_prediction != prediction,
})

```

```

# Define a reusable function that summarizes a table of model predictions.
def prediction_metrics(predictions):

    # Calculate signed error as predicted K% minus actual K%.
    error = predictions["predicted_k_pct"] - predictions["actual_k_pct"]

    # Return the primary and secondary evaluation measurements.
    return pd.Series({

        # Count the player-seasons being evaluated.
        "n": len(predictions),

        # Calculate the average absolute miss, treating every player equally.
        "mae": error.abs().mean(),

        # Calculate RMSE, which gives especially large misses more influence.
        "rmse": np.sqrt(np.mean(error ** 2)),

        # Calculate MAE while giving higher-workload players more influence.
        "tbf_weighted_mae": np.average(error.abs(), weights=predictions["TBF"]),

        # Count predictions that originally fell outside the valid K% range.
        "clipped_predictions": predictions["prediction_was_clipped"].sum(),
    })

```

```

# Tune Ridge's feature group and alpha using only the 2023 forecast.
# Because 2023 is the validation season, the model trains only on 2022 rows.
selection_rows = []

# Test each progressively larger candidate feature set.
for feature_set_name, feature_columns in FEATURE_SETS.items():

    # Test each possible Ridge coefficient-shrinkage strength.
    for alpha in RIDGE_ALPHAS:

        # Create a fresh Ridge pipeline using the current alpha.
        ridge = make_linear_pipeline(Ridge(alpha=alpha))

        # Train on 2022 and predict 2023 using the current features and alpha.
        predictions = predict_validation_season(ridge, feature_columns, 2023, f"Ridge: {feature_set_name}, alpha={alpha}")

        # Calculate the 2023 prediction metrics for this combination.
        metrics = prediction_metrics(predictions)

        # Store the tested features, alpha, and resulting accuracy measurements.
        # ** expands the metrics dictionary into separate columns.
        selection_rows.append({
            "feature_set": feature_set_name,
            "n_features": len(feature_columns),
            "alpha": alpha,
            **metrics.to_dict(),
        })

# Combine all feature-set and alpha results and rank them by 2023 MAE.
ridge_selection_grid = pd.DataFrame(selection_rows).sort_values("mae").reset_index(drop=True)

# Find the best-performing alpha within each feature set.
# This creates a smaller table for comparing the feature groups.
best_by_feature_set = (
    ridge_selection_grid.sort_values("mae")
    .groupby("feature_set", as_index=False)
    .first()
    .sort_values("mae")
    .reset_index(drop=True)
)

# Treat MAE differences within 0.01 percentage points as effectively tied.
SELECTION_TOLERANCE = 0.0001 # 0.01 percentage points of K%.

# Find the lowest 2023 MAE produced by any feature-set and alpha combination.
best_selection_mae = ridge_selection_grid["mae"].min()

# Keep every combination whose MAE is within the allowed distance of the best result.
# Rank tied candidates by fewer features first, then MAE, then alpha.
parsimonious_candidates = ridge_selection_grid.loc[ridge_selection_grid["mae"] <= best_selection_mae + SELECTION_TOLERANCE]

# Select the first row: the simplest model whose MAE is effectively tied for best.
selected_specification = parsimonious_candidates.iloc[0]

```

```

# Store the name of the selected feature group.
SELECTED_FEATURE_SET_NAME = selected_specification["feature_set"]

# Store the selected Ridge penalty as a numeric value.
BEST_RIDGE_ALPHA = float(selected_specification["alpha"])

# Retrieve the actual feature names belonging to the selected group.
SELECTED_FEATURES = FEATURE_SETS[SELECTED_FEATURE_SET_NAME]

# Display the best alpha and accuracy results found for each feature group.
display(best_by_feature_set[["feature_set", "n_features", "alpha", "mae", "rmse", "tbf_weighted_mae"]])

# Print the feature group and alpha selected from the 2023 forecast.
print(f"Selected from 2023: {SELECTED_FEATURE_SET_NAME}, alpha={BEST_RIDGE_ALPHA:g}")

```

	feature_set	n_features	alpha	mae	rmse	tbf_weighted_mae
0	+ age	20	0.1	0.0536	0.0769	0.0364
1	+ league context	22	0.1	0.0536	0.0769	0.0364
2	+ Stuff+	18	100.0	0.0546	0.0782	0.0371
3	+ workload	12	100.0	0.0565	0.0802	0.0374
4	K history	6	100.0	0.0565	0.0800	0.0372

Selected from 2023: + age, alpha=0.1

```

# Apply each feature set's 2023-selected alpha to the untouched 2024 season.
# The 2024 results are used for final evaluation, not further model tuning.
ablation_validation_rows = []

# Loop through the best 2023 alpha found for each candidate feature set.
# itertools allows each result row to be accessed using names such as row.feature_set.
for row in best_by_feature_set.iteruples(index=False):

    # Retrieve the feature columns belonging to the current feature set.
    feature_columns = FEATURE_SETS[row.feature_set]

    # Create a fresh Ridge pipeline using the alpha selected from 2023.
    ridge = make_linear_pipeline(Ridge(alpha=row.alpha))

    # Train on 2022-2023 and predict the untouched 2024 player-seasons.
    predictions = predict_validation_season(ridge, feature_columns, 2024, f"Ridge: {row.feature_set}")

    # Calculate 2024 prediction accuracy for the current feature set.
    metrics = prediction_metrics(predictions)

    # Store the 2023 selection result beside the corresponding 2024 validation result.
    ablation_validation_rows.append({
        "feature_set": row.feature_set,
        "alpha_selected_on_2023": row.alpha,
        "selection_2023_mae": row.mae,
        "validation_2024_mae": metrics["mae"],
    })

```

```

        "validation_2024_rmse": metrics["rmse"],
        "validation_2024_tbf_weighted_mae": metrics["tbf_weighted_mae"],
    })

```

Convert the collected 2024 validation results into a comparison table.

```
feature_ablation_results = pd.DataFrame(ablation_validation_rows)
```

Display how each progressively larger feature set performed in 2023 and 2024.

```
display(feature_ablation_results.round(4))
```

	feature_set	alpha_selected_on_2023	selection_2023_mae	validation_2024_mae	validation_2024_rmse
0	+ age	0.1	0.0536	0.0526	0.0726
1	+ league context	0.1	0.0536	0.0529	0.0730
2	+ Stuff+	100.0	0.0546	0.0526	0.0735
3	+ workload	100.0	0.0565	0.0543	0.0753
4	K history	100.0	0.0565	0.0544	0.0754

Store reusable instructions for creating a fresh version of each fitted model.

OrderedDict preserves the intended OLS, Ridge, and boosting comparison order.

```
MODEL_FACTORIES = OrderedDict({
```

Create an OLS pipeline as the basic fitted linear comparison.

```
"OLS": lambda: make_linear_pipeline(LinearRegression()),
```

Create a Ridge pipeline using the alpha selected from the 2023 forecast.

```
"Ridge": lambda: make_linear_pipeline(Ridge(alpha=BEST_RIDGE_ALPHA)),
```

Create the previously defined nonlinear boosting pipeline.

```
"Gradient boosting": make_boosting_pipeline,
```

```
})
```

Create an empty list to store prediction tables from every model and season.

```
comparison_frames = []
```

Generate predictions from each fitted model.

```
for model_name, model_factory in MODEL_FACTORIES.items():
```

Predict both validation seasons using the same selected feature set.

```
for validation_season in MODEL_VALIDATION_SEASONS:
```

Create a fresh model, train it on earlier seasons, and predict the validation season

A fresh model prevents information from one validation run carrying into another.

```
comparison_frames.append(predict_validation_season(model_factory(), SELECTED_FEATURES,
```

Select two simple benchmarks for comparison with the fitted models.

Prior-season K% represents the simplest baseline; Marcel represents the strongest baseline.

```

benchmark_columns = OrderedDict({
    "Prior-season K%": "prior_season_k_pct",
    "Marcel": "marcel_k_pct",
})

# Prepare each benchmark using the same players and validation seasons as the fitted models.
for benchmark_name, prediction_column in benchmark_columns.items():

    # Select the 2023 and 2024 historical rows and the current benchmark prediction.
    benchmark = baseline_backtest.loc[
        baseline_backtest["target_season"].isin(MODEL_VALIDATION_SEASONS),
        ["PlayerId", "Name", "target_season", "K%", "TBF", prediction_column],
    ].copy()

    # Rename the benchmark columns to match the fitted-model prediction tables.
    # A shared structure allows all results to be combined and evaluated together.
    benchmark = benchmark.rename(columns={
        "target_season": "validation_season",
        "K%": "actual_k_pct",
        prediction_column: "predicted_k_pct",
    })

    # Label every row with the name of the benchmark that produced the prediction.
    benchmark["model"] = benchmark_name

    # Record that baseline predictions did not require clipping.
    benchmark["prediction_was_clipped"] = False

    # Add the completed benchmark table to the fitted-model prediction tables.
    comparison_frames.append(benchmark)

# Combine every fitted-model and benchmark prediction into one detailed table.
model_predictions = pd.concat(comparison_frames, ignore_index=True)

# Calculate accuracy separately for every model in each validation season.
model_metrics_by_year = (
    model_predictions.groupby(["validation_season", "model"], sort=False)
    .apply(prediction_metrics)
    .reset_index()
)

# Calculate each model's overall accuracy across the combined 2023–2024 predictions.
# Rank the models from lowest to highest overall MAE.
model_metrics_overall = (
    model_predictions.groupby("model", sort=False)
    .apply(prediction_metrics)
    .reset_index()
    .sort_values("mae")
    .reset_index(drop=True)
)

# List the error metrics that are currently stored as decimal K% values.
metric_columns = ["mae", "rmse", "tbw_weighted_mae"]

```

```

# Copy the metric tables so display formatting does not change the original results.
model_metrics_by_year_pp = model_metrics_by_year.copy()
model_metrics_overall_pp = model_metrics_overall.copy()

# Convert yearly errors from decimal K% to K percentage points.
model_metrics_by_year_pp[metric_columns] *= 100

# Convert overall errors from decimal K% to K percentage points.
model_metrics_overall_pp[metric_columns] *= 100

# Display each model's results separately for 2023 and 2024.
display(model_metrics_by_year_pp.round(2))

# Display each model's combined 2023-2024 results ranked by MAE.
display(model_metrics_overall_pp.round(2))

```

	validation_season		model	n	mae	rmse	tbw_weighted_mae	clipped_predictions
0	2023		OLS	863.0	5.36	7.69	3.64	1.0
1	2024		OLS	855.0	5.27	7.27	3.57	2.0
2	2023		Ridge	863.0	5.36	7.69	3.64	1.0
3	2024		Ridge	855.0	5.26	7.26	3.57	2.0
4	2023	Gradient boosting		863.0	5.58	7.85	3.80	0.0
5	2024	Gradient boosting		855.0	5.33	7.29	3.62	7.0
6	2023	Prior-season K%		863.0	6.01	8.65	3.82	0.0
7	2024	Prior-season K%		855.0	6.33	9.09	4.00	0.0
8	2023		Marcel	863.0	5.85	8.54	3.33	0.0
9	2024		Marcel	855.0	5.86	8.38	3.34	0.0

		model	n	mae	rmse	tbw_weighted_mae	clipped_predictions
0		Ridge	1718.0	5.31	7.48	3.60	3.0
1		OLS	1718.0	5.31	7.48	3.60	3.0
2	Gradient boosting		1718.0	5.46	7.58	3.71	7.0
3		Marcel	1718.0	5.85	8.46	3.33	0.0
4	Prior-season K%		1718.0	6.17	8.87	3.91	0.0

```

# After completing 2024 validation, refit the selected Ridge specification using all labeled 2023 data.
# This model is used to inspect feature associations, not to recalculate 2024 validation performance.
provisional_ridge_model = make_linear_pipeline(Ridge(alpha=BEST_RIDGE_ALPHA))

# Fit Ridge using the selected features and actual next-season K% outcomes.
provisional_ridge_model.fit(baseline_backtest[SELECTED_FEATURES], baseline_backtest["K%"])

```

```

# Retrieve the feature names remaining after missing-value imputation.
imputed_feature_names = provisional_ridge_model.named_steps["imputer"].get_feature_names_out()

# Retrieve Ridge's coefficient for each standardized feature.
# Standardization makes coefficient magnitudes more comparable across different measurement scales.
standardized_coefficients = provisional_ridge_model.named_steps["model"].coef_

# Create a table matching each selected feature to its Ridge coefficient.
ridge_coefficients = pd.DataFrame({
    "feature": imputed_feature_names,
    "standardized_coefficient": standardized_coefficients,
})

# Calculate coefficient size without considering whether the association is positive or negative.
ridge_coefficients["absolute_coefficient"] = ridge_coefficients["standardized_coefficient"].abs()

# Rank features from the largest to smallest coefficient magnitude.
ridge_coefficients = ridge_coefficients.sort_values("absolute_coefficient", ascending=False)

# Display the 15 features with the largest standardized coefficients.
display(ridge_coefficients.head(15).round(4))

# Clarify that coefficients describe predictive relationships rather than cause and effect.
print("Coefficients describe association after controlling for the other included features; they do not describe causation.")

```

	feature	standardized_coefficient	absolute_coefficient
10	no_observed_history	0.0580	0.0580
5	marcel_reliability	-0.0431	0.0431
7	log1p_recent_tbf	0.0423	0.0423
8	log1p_available_history_tbf	0.0358	0.0358
13	most_recent_stuff_plus	0.0263	0.0263
9	years_since_last_observed	0.0210	0.0210
16	missing_prior_stuff_plus	-0.0194	0.0194
11	missed_prior_season	-0.0194	0.0194
4	marcel_k_pct	0.0192	0.0192
18	age_centered	-0.0131	0.0131
2	available_history_k_pct	0.0092	0.0092
6	log1p_prior_tbf	0.0078	0.0078
19	age_centered_squared	0.0047	0.0047
3	recent_k_pct	-0.0043	0.0043
12	prior_stuff_plus	-0.0031	0.0031

Coefficients describe association after controlling for the other included features; they are not causal effects.

```
# Compare Ridge and Marcel across pre-season history groups.
diagnostic_context = baseline_backtest[[
    "PlayerId", "target_season", "prior_tbf", "no_observed_history",
]].rename(columns={"target_season": "validation_season"})
diagnostic_predictions = model_predictions.loc[
    model_predictions["model"].isin(["Ridge", "Marcel"])
].merge(diagnostic_context, on=["PlayerId", "validation_season"], how="left")

diagnostic_predictions["history_group"] = np.select(
    [
        diagnostic_predictions["no_observed_history"].eq(1),
        diagnostic_predictions["prior_tbf"].eq(0),
        diagnostic_predictions["prior_tbf"].lt(50),
        diagnostic_predictions["prior_tbf"].lt(200),
    ],
    ["No prior history", "Missed prior season", "1-49 prior TBF", "50-199 prior TBF"],
    default="200+ prior TBF",
)
history_group_metrics = (
    diagnostic_predictions.groupby(["history_group", "model"], sort=False)
    .apply(prediction_metrics)
    .reset_index()
)
display(history_group_metrics.round(4))

# Count each validation pitcher-season once when describing history volume.
history_volume = diagnostic_predictions.loc[
    diagnostic_predictions["model"].eq("Ridge"), ["prior_tbf"]
].copy()
history_volume["history_volume"] = np.where(
    history_volume["prior_tbf"].ge(200), "200+ prior TBF", "Under 200 or incomplete"
)
history_volume_summary = (
    history_volume.groupby("history_volume", as_index=False)
    .size()
    .rename(columns={"size": "pitcher_seasons"})
)
history_volume_summary["share"] = (
    history_volume_summary["pitcher_seasons"] / history_volume_summary["pitcher_seasons"].sum()
)
display(history_volume_summary)

# Isolate the requested same-sample benchmark comparison in percentage points.
same_sample_models = ["Prior-season K%", "Marcel", "Ridge"]
same_sample_comparison = (
    model_metrics_overall.set_index("model")
    .loc[same_sample_models, ["n", "mae", "rmse", "tbf_weighted_mae"]]
    .copy()
)
same_sample_comparison[["mae", "rmse", "tbf_weighted_mae"]] *= 100
prior_mae_pp = same_sample_comparison.loc["Prior-season K%", "mae"]
same_sample_comparison["mae_improvement_vs_prior_pp"] = (
    prior_mae_pp - same_sample_comparison["mae"]
)
```

```
)
display(same_sample_comparison.round(2))

ridge_overall = model_metrics_overall.set_index("model").loc["Ridge"]
marcel_overall = model_metrics_overall.set_index("model").loc["Marcel"]
prior_overall = model_metrics_overall.set_index("model").loc["Prior-season K%"]
print(f"Ridge reduced unweighted MAE by {(marcel_overall['mae'] - ridge_overall['mae']) * 100}%")
print(f"Ridge changed TBF-weighted MAE by {(ridge_overall['tbf_weighted_mae'] - marcel_overall['tbf_weighted_mae']) * 100}%")
print(f"Ridge reduced MAE versus prior-season K% by {(prior_overall['mae'] - ridge_overall['mae']) * 100}%")
```

	history_group	model	n	mae	rmse	tbf_weighted_mae	clipped_predictions
0	No prior history	Ridge	401.0	0.0844	0.1090	0.0614	0.0
1	1–49 prior TBF	Ridge	192.0	0.0566	0.0773	0.0424	1.0
2	Missed prior season	Ridge	102.0	0.0556	0.0744	0.0457	2.0
3	200+ prior TBF	Ridge	653.0	0.0340	0.0459	0.0283	0.0
4	50–199 prior TBF	Ridge	370.0	0.0504	0.0690	0.0407	0.0
5	No prior history	Marcel	401.0	0.0931	0.1217	0.0482	0.0
6	1–49 prior TBF	Marcel	192.0	0.0838	0.1132	0.0409	0.0
7	Missed prior season	Marcel	102.0	0.0651	0.0896	0.0382	0.0
8	200+ prior TBF	Marcel	653.0	0.0335	0.0452	0.0277	0.0
9	50–199 prior TBF	Marcel	370.0	0.0503	0.0686	0.0391	0.0

	history_volume	pitcher_seasons	share
0	200+ prior TBF	653	0.380093
1	Under 200 or incomplete	1065	0.619907

	n	mae	rmse	tbf_weighted_mae	mae_improvement_vs_prior_pp
model					
Prior-season K%	1718.0	6.17	8.87	3.91	0.00
Marcel	1718.0	5.85	8.46	3.33	0.31
Ridge	1718.0	5.31	7.48	3.60	0.85

Ridge reduced unweighted MAE by 0.54 percentage points.

Ridge changed TBF-weighted MAE by +0.27 percentage points.

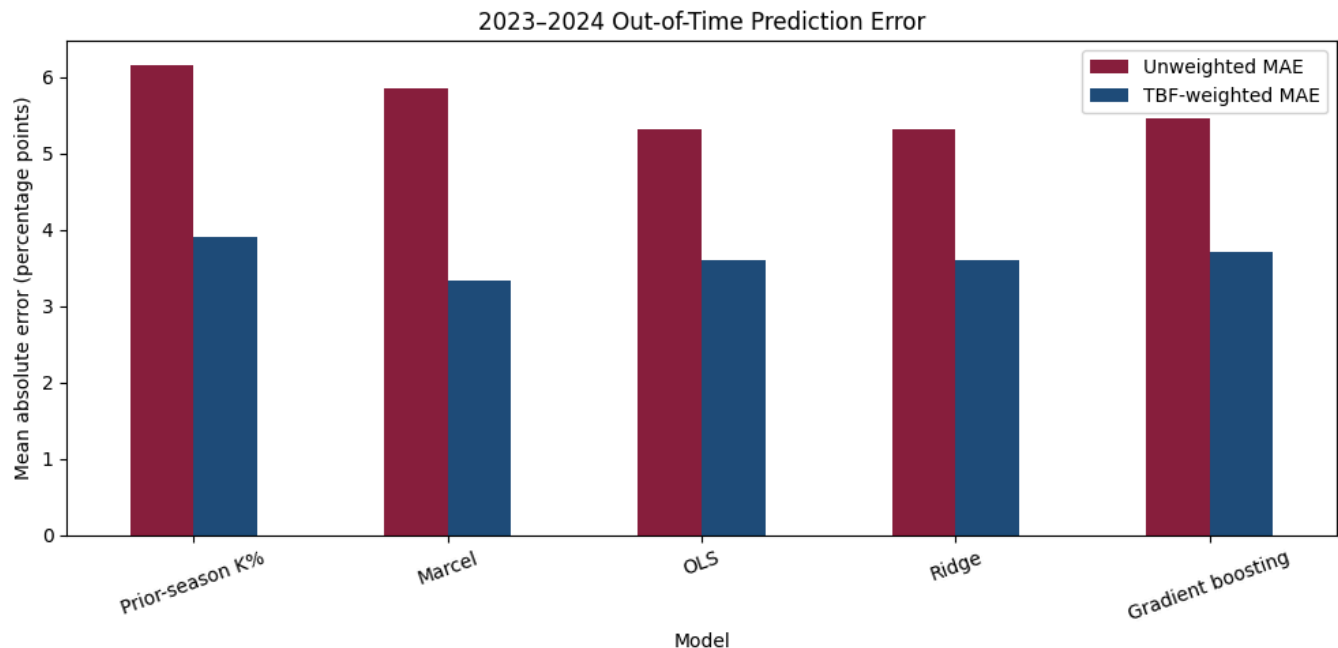
Ridge reduced MAE versus prior-season K% by 0.85 percentage points on the same rows.

```
# Show the tradeoff between equal-player and workload-weighted accuracy.
plot_metrics = model_metrics_overall.set_index("model").loc[
    ["Prior-season K%", "Marcel", "OLS", "Ridge", "Gradient boosting"],
    ["mae", "tbf_weighted_mae"],
] * 100
ax = plot_metrics.plot.bar(figsize=(10, 5), color=["#8B1E3F", "#1F4E79"])
```

```

ax.set_title("2023-2024 Out-of-Time Prediction Error")
ax.set_xlabel("Model")
ax.set_ylabel("Mean absolute error (percentage points)")
ax.legend(["Unweighted MAE", "TBF-weighted MAE"])
ax.tick_params(axis="x", rotation=20)
plt.tight_layout()
plt.show()

```



Same-sample and visual diagnostics

The prior-season benchmark is evaluated on exactly the same 2023–2024 pitcher-seasons as Ridge and Marcel. For a pitcher without a recorded season at \$t-1\$, the feature builder substitutes the prior league K%; this keeps the comparison complete instead of silently dropping the hardest cases. A same-sample comparison ensures that an apparent improvement cannot be caused by evaluating one method on an easier group of pitchers.

The predicted-versus-actual scatterplot shows individual pitcher-season errors. Each dot compares one pitcher's prediction with his actual K% in the same season; the colors identify whether that season was 2023 or 2024, rather than comparing one year with the other. The horizontal position is actual K%, and the vertical position is predicted K%. A dot at \$(30, 25)\$ therefore means 30% actual and 25% predicted, an underprediction of 5 percentage points. A point on the dashed 45-degree line is exact, a point above it is an overprediction, and a point below it is an underprediction. Greater distance from the line means a larger error. Extreme actual rates are often low-TBF outcomes, while sensible projections should be less extreme because they regress uncertain observations toward typical values.

The calibration plot groups Ridge predictions into ten equally sized bins. Within each bin, mean predicted K% is compared with mean actual K%. A calibrated model should follow the 45-degree line on average even though individual outcomes remain noisy. Both plots use historical out-of-time predictions only; neither uses 2025 performance.

```

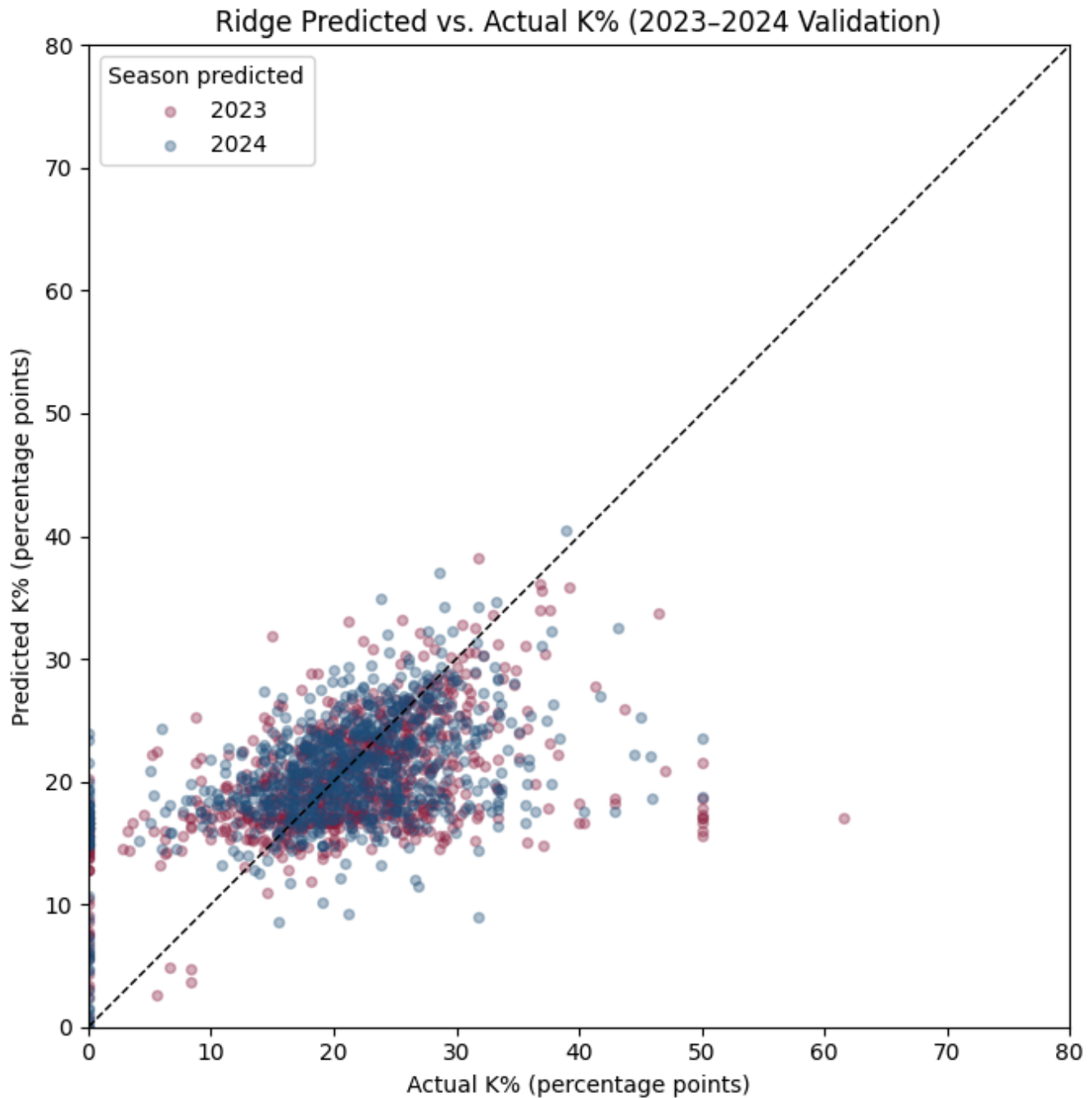
# Compare each Ridge prediction with that pitcher's actual K%.
ridge_plot = model_predictions.loc[model_predictions["model"].eq("Ridge")].copy()
season_colors = {2023: "#8B1E3F", 2024: "#1F4E79"}

fig, ax = plt.subplots(figsize=(7, 7))
for season in MODEL_VALIDATION_SEASONS:
    season_data = ridge_plot.loc[ridge_plot["validation_season"].eq(season)]
    ax.scatter(
        season_data["actual_k_pct"] * 100,
        season_data["predicted_k_pct"] * 100,
        s=18, alpha=0.35, color=season_colors[season], label=str(season),
    )

ax.plot([0, 80], [0, 80], color="black", linestyle="--", linewidth=1)
ax.set(xlim=(0, 80), ylim=(0, 80))
ax.set_aspect("equal", adjustable="box")
ax.set_title("Ridge Predicted vs. Actual K% (2023-2024 Validation)")
ax.set_xlabel("Actual K% (percentage points)")
ax.set_ylabel("Predicted K% (percentage points)")
ax.legend(title="Season predicted")

# Disclose any extreme outcomes omitted by the requested axis limit.
outside_view = (
    ridge_plot["actual_k_pct"].gt(0.80)
    | ridge_plot["predicted_k_pct"].gt(0.80)
).sum()
if outside_view:
    ax.text(2, 76, f"{outside_view} point(s) above 80% not shown", fontsize=8)
plt.tight_layout()
plt.show()

```



```
# Average outcomes within ten equally sized Ridge prediction groups.
ridge_calibration = ridge_plot.copy()
ridge_calibration["prediction_bin"] = pd.qcut(
    ridge_calibration["predicted_k_pct"], q=10, labels=False, duplicates="drop"
)
calibration_table = (
    ridge_calibration.groupby("prediction_bin", as_index=False)
    .agg(
        n=("actual_k_pct", "size"),
        mean_predicted_k_pct=("predicted_k_pct", "mean"),
        mean_actual_k_pct=("actual_k_pct", "mean"),
    )
)
calibration_display = calibration_table.copy()
calibration_display[["mean_predicted_k_pct", "mean_actual_k_pct"]] *= 100
```

```

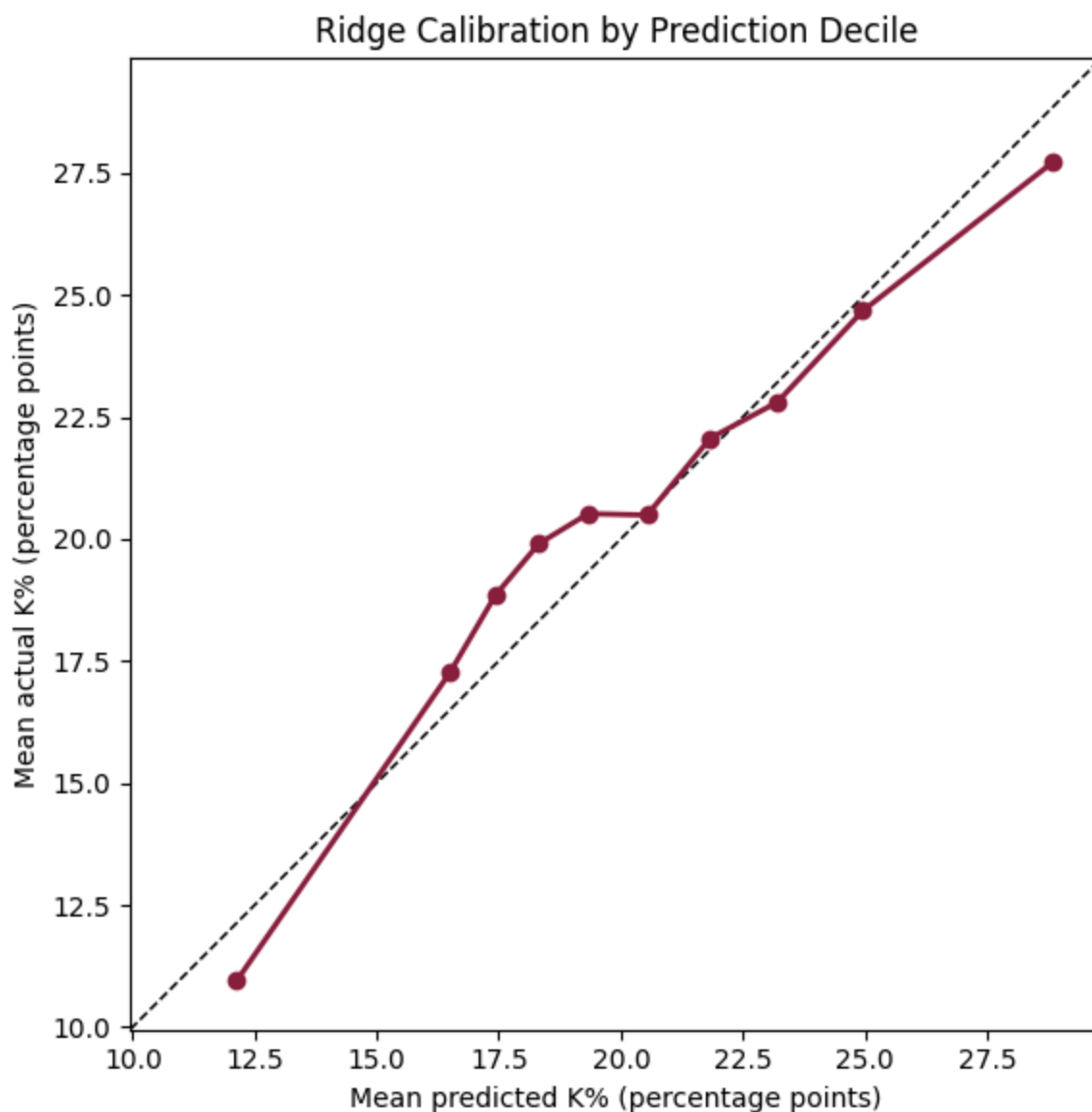
display(calibration_display.round(2))

predicted_pp = calibration_table["mean_predicted_k_pct"] * 100
actual_pp = calibration_table["mean_actual_k_pct"] * 100
axis_low = min(predicted_pp.min(), actual_pp.min()) - 1
axis_high = max(predicted_pp.max(), actual_pp.max()) + 1

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot([axis_low, axis_high], [axis_low, axis_high], color="black", linestyle="--", linewidth=2)
ax.plot(predicted_pp, actual_pp, marker="o", color="#8B1E3F", linewidth=2)
ax.set(xlim=(axis_low, axis_high), ylim=(axis_low, axis_high))
ax.set_aspect("equal", adjustable="box")
ax.set_title("Ridge Calibration by Prediction Decile")
ax.set_xlabel("Mean predicted K% (percentage points)")
ax.set_ylabel("Mean actual K% (percentage points)")
plt.tight_layout()
plt.show()

```

	prediction_bin	n	mean_predicted_k_pct	mean_actual_k_pct
0	0	172	12.13	10.95
1	1	200	16.51	17.27
2	2	155	17.43	18.85
3	3	160	18.32	19.91
4	4	182	19.33	20.52
5	5	162	20.54	20.49
6	6	171	21.83	22.06
7	7	172	23.20	22.80
8	8	172	24.95	24.67
9	9	172	28.85	27.72



Model comparison findings

- Selected Ridge specification:** 2023 tuning selected the 20-feature model containing K% history, workload and availability, Stuff+, and centered age terms, with $\alpha=0.1$. Adding two league-context features improved MAE by less than the prespecified 0.01-percentage-point threshold, so the parsimony rule favored the simpler model.
- Out-of-time 2024 result:** Ridge produced a 5.26-percentage-point MAE and 7.26-point RMSE, compared with Marcel's 5.86-point MAE and 8.38-point RMSE. Thus, the fitted model improved both primary metrics in the season that was not used for feature or penalty selection.
- Combined 2023–2024 result:** Ridge reduced unweighted MAE from 5.85 to 5.31 percentage points, an improvement of 0.54 points. Gradient boosting reached 5.46 points and did not justify its added complexity. OLS nearly matched Ridge, but Ridge is preferred because coefficient shrinkage provides protection against the substantial overlap among the historical-rate features.
- Same-sample prior-year comparison:** across the same 1,718 validation pitcher-seasons, prior-season K% produced a 6.17-point MAE versus Ridge's 5.31 points. Ridge therefore improved MAE by

0.85 percentage points without benefiting from an easier evaluation sample.

- **Visual diagnostics:** individual actual K% values are much more dispersed than Ridge predictions, especially for small workloads; this is the expected effect of regression toward the mean. The calibration deciles follow the 45-degree line reasonably closely. Ridge mildly underpredicted several middle bins and mildly overpredicted the lowest and highest bins, but the average gaps were small relative to individual-season noise.
- **Important weighting tradeoff:** Marcel retained the better TBF-weighted MAE, 3.33 versus 3.60 percentage points. Ridge's advantage was concentrated among difficult low-information players: for players with 1–49 prior TBF, Ridge reduced unweighted MAE from 8.38 to 5.66 points. Marcel remained slightly better for players with at least 200 prior TBF, 3.35 versus 3.40 points. Limited or incomplete prior history represented 1,065 of 1,718 validation pitcher-seasons (62%), versus 653 (38%) with at least 200 prior TBF. Low TBF should not be equated automatically with relief pitching; it can also reflect injury, a midseason debut, or occasional position-player pitching.
- **Why not switch models at a TBF cutoff?** A segmented model is valid in principle, but choosing a cutoff from the combined 2023–2024 results and reporting performance on those same seasons would overfit the validation evidence. The 50–199 TBF group was essentially tied and Marcel's 200+ advantage was only 0.05 percentage points of unweighted MAE. Ridge also already receives Marcel K%, Marcel reliability, and workload features, allowing it to use that information continuously. With only two validation seasons, the small possible gain did not justify a new hard rule.
- **Feature interpretation:** Stuff+ and age improved the 2023 selection result, and the 2024 feature-set comparison remained close. Individual coefficients should not be interpreted causally or in isolation because several predictors describe overlapping aspects of the same pitching history.

How Marcel is used: Marcel serves two roles: it is evaluated as a standalone benchmark, and its projected K% and reliability are also supplied as features to Ridge. The final prediction is produced by Ridge, which learns how much Marcel information to use alongside the other features.

Model decision: use Ridge for the 2025 case-study projections because unweighted MAE matches the one-prediction-per-player objective. Retain Marcel as a transparent benchmark and document that it would be preferred if high-workload accuracy were the primary objective. A pre-2025 swinging-strike metric remains a possible challenger rather than an assumed improvement. The 2025 outcomes remain sealed.

Role scope control: starter/reliever role was considered because role can affect strikeout rates, but the dataset does not include games started, appearances, or innings. TBF alone cannot reliably distinguish relievers from starters, injured pitchers, or midseason call-ups, and actual 2025 roles would not be permissible pre-season information. Rather than introduce a potentially misleading classification, the model retains workload and history indicators; a future extension could construct prior-season role from pre-2025 GS/G and innings data, with a separate mixed-role category.

Final 2025 predictions

The model specification is now frozen: Ridge with the feature set and penalty selected using the historical process above. The final model is refit on all 2,589 eligible 2022–2024 target player-seasons, allowing the imputer, scaler, and Ridge coefficients to learn from every permissible training example. It then scores the 873-player 2025 feature table built only from performance through 2024 and target-season age.

Refitting after model selection is standard because the 2024 outcomes can improve the final coefficient estimates once they are no longer being used to choose among specifications. No 2025 K%, TBF, Stuff+, or team information enters the model. The exported K% values use the same decimal scale as the source file: for example, `0.250000` means 25.00%.

```
# Development assistance: OpenAI Codex supported code structure and review.
# https://openai.com/codex/

# Refit the frozen Ridge specification on every permissible target season.
final_ridge_model = make_linear_pipeline(Ridge(alpha=BEST_RIDGE_ALPHA))
final_ridge_model.fit(baseline_backtest[SELECTED_FEATURES], baseline_backtest["K%"])

raw_2025_prediction = final_ridge_model.predict(features_2025[SELECTED_FEATURES])
final_2025_prediction = np.clip(raw_2025_prediction, 0.0, 1.0)

# Preserve the source player order and express K% on the source decimal scale.
predictions_2025 = features_2025[["PlayerId", "Name"]].copy()
predictions_2025.insert(0, "Season", 2025)
predictions_2025["Predicted K%"] = final_2025_prediction

# Final integrity checks prevent missing, duplicate, or illogical projections.
assert len(predictions_2025) == len(players_2025)
assert predictions_2025["PlayerId"].is_unique
assert set(predictions_2025["PlayerId"]) == set(players_2025["PlayerId"])
assert predictions_2025["Predicted K%"].notna().all()
assert predictions_2025["Predicted K%"].between(0.0, 1.0).all()

output_dir = project_root / "outputs"
output_dir.mkdir(exist_ok=True)
prediction_output_path = output_dir / "k_2025_predictions.csv"
predictions_2025.to_csv(prediction_output_path, index=False, float_format="%.6f")

projection_audit = pd.Series({
    "training_player_seasons": len(baseline_backtest),
    "projected_players": len(predictions_2025),
    "minimum_prediction_pct": predictions_2025["Predicted K%"].min() * 100,
    "median_prediction_pct": predictions_2025["Predicted K%"].median() * 100,
    "maximum_prediction_pct": predictions_2025["Predicted K%"].max() * 100,
    "clipped_predictions": int((raw_2025_prediction != final_2025_prediction).sum()),
})
display(projection_audit.to_frame("value").round(2))
print(f"Saved final predictions to: {prediction_output_path.relative_to(project_root)}")
```

	value
training_player_seasons	2589.00
projected_players	873.00
minimum_prediction_pct	0.00
median_prediction_pct	20.65
maximum_prediction_pct	38.64
clipped_predictions	5.00

Saved final predictions to: outputs\k_2025_predictions.csv

Final projection graphics

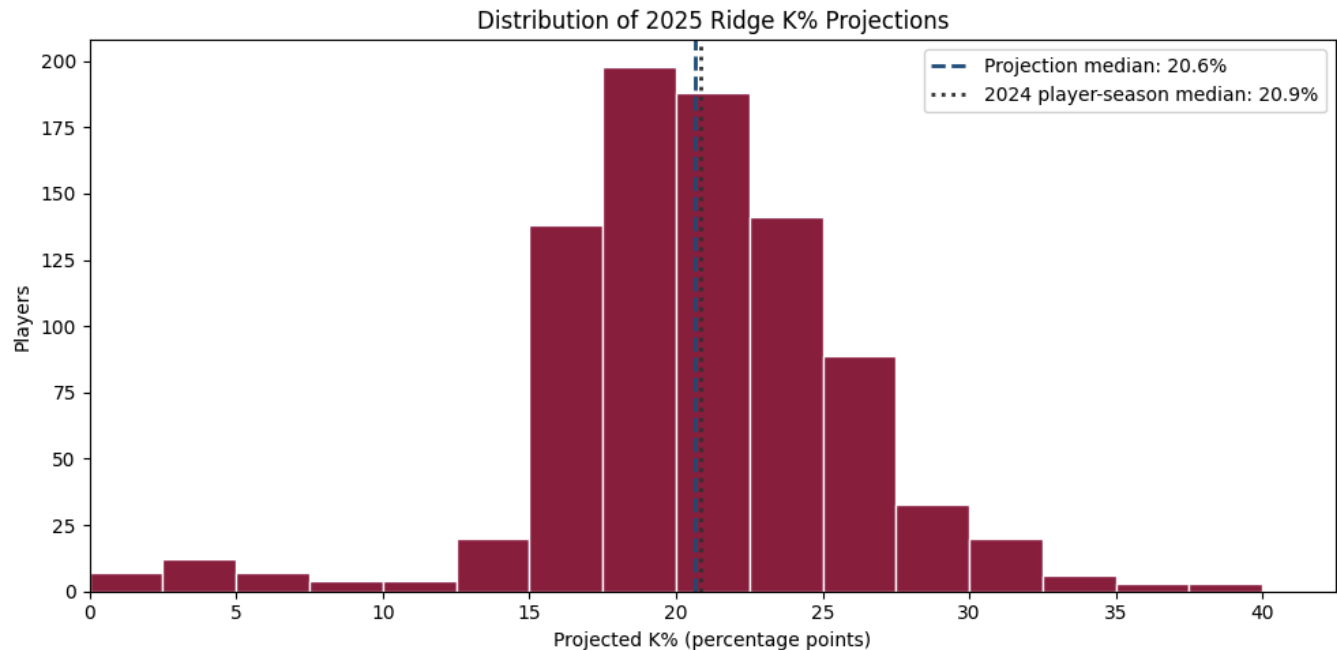
The distribution chart is a final plausibility check. It shows the spread of all 873 projections and compares the unweighted 2025 projection median with the comparable unweighted 2024 player-season median. The projection distribution should be narrower than the distribution of observed single-season K% because the model deliberately regresses noisy and limited histories toward typical values. The TBF-weighted 2024 league K% is not used as the chart reference because it answers a different question by giving established, high-workload pitchers much more influence.

The top-20 chart translates the model into an intuitive baseball result. It ranks **projected K% only**, not overall pitcher quality, expected workload, or total strikeouts. Bar colors show how much immediately prior history is available; this is important because a high-rate forecast for a pitcher with limited or missing 2024 history carries more uncertainty than the same forecast for an established pitcher.

```
# Compare the center and spread of all final 2025 projections.
projection_pct = predictions_2025["Predicted K%"] * 100
median_projection_pct = projection_pct.median()
historical_2024_median_pct = (
    development_df.loc[development_df["Season"].eq(2024), "K%"].median() * 100
)

fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(projection_pct, bins=np.arange(0, 42.5, 2.5), color="#8B1E3F", edgecolor="white")
ax.axvline(
    median_projection_pct, color="#1F4E79", linestyle="--", linewidth=2,
    label=f"Projection median: {median_projection_pct:.1f}%",
)
ax.axvline(
    historical_2024_median_pct, color="#333333", linestyle=":", linewidth=2,
    label=f"2024 player-season median: {historical_2024_median_pct:.1f}%",
)
ax.set_title("Distribution of 2025 Ridge K% Projections")
ax.set_xlabel("Projected K% (percentage points)")
ax.set_ylabel("Players")
ax.set_xlim(0, 42.5)
ax.legend()
```

```
plt.tight_layout()
plt.show()
```



```
from matplotlib.patches import Patch

# Add pre-season history context to the highest projected K% values.
projection_context = features_2025[[
    "PlayerId", "prior_tbf", "no_observed_history", "missed_prior_season",
]].copy()
top_20 = predictions_2025.merge(projection_context, on="PlayerId", how="left")
top_20["history_group"] = np.select(
    [
        top_20["no_observed_history"].eq(1),
        top_20["missed_prior_season"].eq(1),
        top_20["prior_tbf"].lt(50),
        top_20["prior_tbf"].lt(200),
    ],
    ["No prior history", "Missed 2024", "1-49 prior TBF", "50-199 prior TBF"],
    default="200+ prior TBF",
)
top_20 = top_20.nlargest(20, "Predicted K%").sort_values("Predicted K%")

history_colors = {
    "No prior history": "#7F7F7F",
    "Missed 2024": "#C98B2E",
    "1-49 prior TBF": "#D95F5F",
    "50-199 prior TBF": "#5B8FF9",
    "200+ prior TBF": "#1F4E79",
}

bar_colors = top_20["history_group"].map(history_colors)

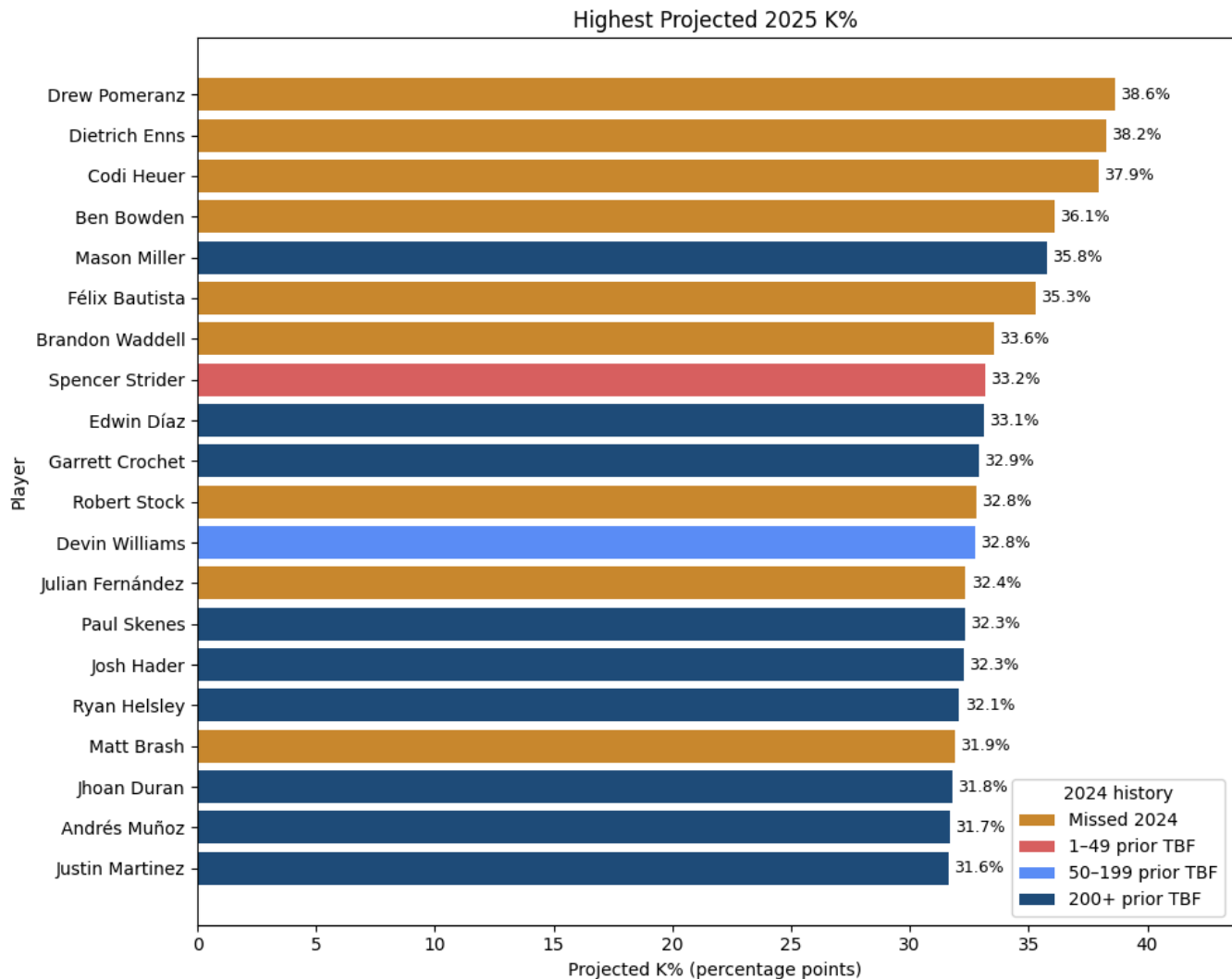
fig, ax = plt.subplots(figsize=(10, 8))
bars = ax.barh(top_20["Name"], top_20["Predicted K%"] * 100, color=bar_colors)
ax.bar_label(
    bars, labels=[f"{value:.1f}%" for value in top_20["Predicted K%"] * 100],
    padding=3, fontsize=9,
```

```

)
ax.set_title("Highest Projected 2025 K%")
ax.set_xlabel("Projected K% (percentage points)")
ax.set_ylabel("Player")
ax.set_xlim(0, top_20["Predicted K%"].max() * 100 + 5)

present_groups = [group for group in history_colors if group in set(top_20["history_group"])]
legend_handles = [
    Patch(facecolor=history_colors[group], label=group) for group in present_groups
]
ax.legend(handles=legend_handles, title="2024 history", loc="lower right")
plt.tight_layout()
plt.show()

```



Limitations and possible improvements

1. **Short validation history.** The dataset begins in 2021, leaving 2023 for specification selection and only 2024 as the untouched final validation season. Results from two forecast years may not generalize perfectly across run environments or changes in league strategy.
2. **Truncated pitcher histories.** Available-history features summarize only the recorded seasons before each target year, not a pitcher's complete MLB career. Earlier MLB performance, minor-league data,

injuries, and offseason changes are unavailable. This is especially important for pitchers with no or little recorded history.

3. **Role and context are not modeled directly.** TBF is not synonymous with starter or reliever status. The model does not explicitly know future role, expected workload, handedness, pitch mix, velocity, team defense, park, catcher, or opponent quality. Target-season TBF cannot be known at projection time.
4. **Position-player pitching complicates league regression.** Regressing a small or missing sample toward league-average pitcher K% assumes that the player belongs to the pitcher population. That is reasonable for an inexperienced pitcher but can overproject a position player making emergency appearances. Ridge can partially learn from low workload, missingness, prior K%, and Stuff+, but it cannot reliably distinguish a rookie pitcher from a position player when neither has supplied pitching history. A future model could add a primary-position indicator known before 2025 and estimate a separate position-player pitching baseline.
5. **The evaluation objective matters.** Ridge leads on equal-player MAE, while Marcel leads on TBF-weighted MAE. A high-workload-focused application could rationally prefer Marcel. A TBF-gated blend was not adopted because the apparent cutoff advantage was small, the 50–199 TBF group was tied, and the short validation window could make a post hoc cutoff unstable.
6. **Stuff+ is useful but incomplete.** Stuff+ measures physical pitch characteristics and captures a stable part of pitch quality, but it does not directly represent every path to a strikeout, including location, called strikes, count management, and batter interaction. Location+ or Pitching+ could add process information, but they should be tested for incremental rather than assumed value.

Why swinging-strike rate is the first external-data candidate

Swinging-strike rate (SwStr%) is swinging strikes divided by total pitches. Whiff% is swinging strikes divided by swings. SwStr% is the preferred first candidate because its total-pitch denominator combines how often a pitcher induces swings with how often those swings miss; it therefore measures missed bats produced per pitch, a process closely connected to strikeout creation. Whiff% isolates bat-missing ability conditional on a swing, but it can overlook whether the pitcher induces enough swings for those misses to accumulate. Neither measure captures called strikes or the full process of reaching and finishing two-strike counts.

A proper SwStr% test would add prior-year, recency- and pitch-weighted, change, and missingness features using only pre-target-season data. The feature design would be selected on 2023 and evaluated once on 2024. It should be retained only if it produces a meaningful, consistent improvement over Ridge because prior K% and Stuff+ may already contain much of the same information. External data are not added to the current scope: the clean, reproducible core model is stronger than a late feature whose collection and player-season joins have not been fully validated. SwStr% is the first extension to test, followed by Whiff%, Pitching+/Location+, and then BB%.

Definitions and context: <https://library.fangraphs.com/pitching/plate-discipline-o-swing-z-swing-etc/> and <https://library.fangraphs.com/pitching/stuff-location-and-pitching-primer/>

Reproducibility and citations

Run all cells from top to bottom with the project root or `notebooks/` as the working directory. The notebook reads `data/k_2026.csv`, verifies the cutoff and feature integrity, reconstructs every historical fold, refits the selected model, and writes `outputs/k_2025_predictions.csv`. Package versions are recorded in `environment.yml`.

Method and software references:

- MAE vs RMSE definitions: <https://www.statology.org/mae-vs-rmse/>
- Marcel projection framework: <https://tangotiger.net/marcel/>
- FanGraphs Stuff+, Location+, and Pitching+ primer: <https://library.fangraphs.com/pitching/stuff-location-and-pitching-primer/>
- FanGraphs plate-discipline and SwStr% definitions: <https://library.fangraphs.com/pitching/plate-discipline-o-swing-z-swing-etc/>
- scikit-learn linear models: https://scikit-learn.org/stable/modules/linear_model.html
- scikit-learn histogram gradient boosting: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html>
- scikit-learn imputation: <https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>
- scikit-learn scaling: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- OpenAI Codex development assistance: <https://openai.com/codex/>